

C++

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

</>

C++

ПРОГРАММИРОВАНИЕ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИИ



J A V A

Урок № 1

Введение в язык программирования Java

Содержание

1. Введение в технологию Java	4
1.1. Базовые понятия технологии Java	8
1.2. Жизненный цикл, компиляции и исполнения приложения Java.....	9
1.3. Установка платформы Java.....	10
1.4. Пример написания первого приложения.....	17
1.5. Обзор существующих IDE	20
2. Базовые конструкции	37
2.1. Понятие о строгой типизации	37
2.2. Типы данных.....	37
2.3. Комментарии	48
2.4. Переменные	50
2.5. Константы	54
2.11. Ввод-вывод в консольной программе	56

2.12. Операции.....	57
2.13. Управляющие конструкции	67
2.9. Методы.....	79
2.10. Основные положения Java Code Convention.....	87
2.11. Отладка программ в Eclipse	91
3. Массивы и строки	97
3.14. Массивы.....	97
3.15. Строки.....	109
Домашняя работа.....	137

1. Введение в технологию Java

В современном мире существует множество различных операционных систем (Windows, Mac OS, Unix-подобные). Программное обеспечение написанное для одной системы не будет работать на другой операционной системе. Это связано с различным устройством исполняемых файлов и функций операционных систем.

Java позволяет решить проблему запуска одного приложения на различных ОС предлагая пользователям не только язык разработки, но и целую платформу.

Этапы развития технологии Java

Январь 1991 г. Начало работы над проектом Stealth	Июнь 1991 г. Гослинг начал работу над компилятором Oak	1992 г. Основание FirstPreson	Апрель 1993 г. Выпущен NCSA Mosaic 1.0	Апрель 1994 г. Основание Sun Interactive
Июнь 1994 г. Начало работы над проектом LiveOak	Сентябрь 1994 г. Впервые представлен продукт HotJava	Май 1995 г. Компания Sun представила Java, HotJava и SunWorld 95 Июнь	Декабрь 1995 г. Sun и Netscape объявили о создании JavaScript	Июнь 1996 г. Выпущен пакет JDK 1.0
1996 г. Объявлены JavaBeans и сервлеты	Конец 1996 г. Объявлены JavaCard и 100% Pure Java	Январь-февраль 1997 г. реализована поддержка JavaBeans и JDK 1.1	Весна 1997 г. Выпущены средства JNDI	Весна 1997 г. Выпущены Java Media API и JDK для Linux

Март 1998 г. Выпущен пакет Swing; объявлен продукт Personal Java for Win CE	Апрель 1998 г. Начались поставки Java Plugin	Декабрь 1998 г. Началось распространение Java 2	Январь 1999 г. Объявлена поддержка службы JNI	Февраль 1999 г. Опубликованы исходные коды Java 2 M
Март 1999 г. Объявлена поддержка XML	Июнь 1999 г. Выпущены средства поддержки JSP	Июнь 1999 г. Разделены направления J2EE, J2SE и J2ME	Декабрь 1999 г. Начались поставки J2EE и J2SE для Linux	Февраль 2000 г. Начались поставки Java API for XML
Декабрь 1999 г. – май 2000 г. Начались поставки J2SE 1.3	Конец 2000 г. Множество производителей оборудования объявили о поддержке платформ на базе J2ME	2001 г. Выпущен пакет J2EE 1.3	2002 г. Выпущен пакет J2SE 1.3	

Цели и задачи технологии Java

Основной целью технологии java является принцип — «Написанное один раз — работает везде» (англ «*Write once — run anywhere.*» WORA). Идея состоит в том, что написав один раз код приложения, можно повторно его использовать на различных устройствах и операционных системах, без каких-либо дополнительных усилий.

Не маловажной задачей java является обеспечение высокой эффективности приложений, что достигается быстрым преобразованием байт-кода в язык машинных кодов.

Большое внимание в java уделяется безопасности. В платформе присутствует множество классов для шифрования и работе с сертификатами.

Язык Java задумывался как «язык для домохозяек», т.е. он очень простой для понимания и изучения.

Сравнение технологии Java с другими технологиями программирования (.Net Framework, C++ и др.)

При разработке Java, за основу был взят язык C++, этим обусловлено множество совпадений в синтаксисе и семантике. За основу объектной модели взят подход из языка Smalltalk.

Сравнение C++ и Java:

- в Java отсутствует перегрузка операторов в классах пользователя;
- в Java отсутствует прямой доступ к памяти через указатель;
- в Java отсутствуют деструкторы;
- в Java отсутствует возможность процедурного программирования;
- в Java статическая длина массивов и проверка индекса при доступе к элементу;
- в Java значение переменных экземпляра (полей класса), в случае отсутствия инициализации, принимают значения по умолчанию (для целых и дробных типов это 0, для boolean — это false, для ссылочных типов — это null);
- в Java отсутствуют препроцессорные операции;
- в Java отсутствуют файлы заголовки (*header*);
- в Java контроль за расходом памяти возложен на специальный механизм сборки мусора, в C++ за этим следит сам программист.

Платформы Java и ее составные части

Java платформа — это набор средств для разработки на языке Java, которые поставляются разработчику в виде единого готового решения. В платформу входят:

- стандартные библиотеки;
- средств компиляции;
- средства выполнения кода;
- средства подготовки документации по коду;
- примеры;
- другие средства для разработки.

Редакции платформ Java

Java платформа поставляется в виде единого пакета разной редакции (комплектации). Для каждой редакции существует отдельная нумерация версий платформы.

JSE или J2SE (*Java Standard Edition*) — это стандартная редакция платформы Java, используемая при разработке консольных приложений, в виде апплетов или standalone приложений с графическим интерфейсом, приложений работающих с базами данных и сетью.

JEE или J2EE (*Java Enterprise Edition*) — это редакция платформы которая предназначена для разработки распределенных приложений уровня предприятия. Содержит в себе технологии Servlet, JSP, EJB.

JME (*Java Micro Edition*) — это редакция платформы которая предназначена для разработки приложений под микрокомпьютеры (мобильные телефоны, планшеты и т.д.). В нее входят облегченные стандартные классы и классы для написания мидлетов (Midlets).

Java FX — это платформа (2007 г.) для разработки приложений с графическим интерфейсом. Включена в 8 JSE, до этого была отдельной библиотекой.

1.1. Базовые понятия технологии Java

Виртуальная машина (JVM) — это спецификация, описывающая абстрактную машину, в которой могут выполняться приложения на java. Машина считается виртуальной, т.к. большую ее часть составляют различные программные компоненты. Существуют JVM реализованные различными производителями, но все они придерживаются единой спецификации. Реализация JVM это программное обеспечение интерпретирующее инструкции байт-кода программы в команды операционной системы.

Компилятор — это средство, которое преобразует исходный код в байт-код. Java компилятор написан на языке java. Существуют компиляторы которые преобразуют исходный код с других языков (Ada, JavaScript, Python, Ruby) в байт-код. Существуют так же языки адаптированные для работы на виртуальной java машине (Scala, Groovy).

Байт код — это набор инструкций на промежуточном языке, предназначенный для выполнения виртуальной машинной. Каждая инструкция кодируется одним байтом. Существует 205 инструкций, остальные (51) являются резервом. Байт-код создается в результате компиляции кода программы, находящейся в файлах с расширением java. Байт-код сохраняется в файлах с расширением class.

Основным принципом технологии java является тезис «Написанное один раз — работает везде.» (WORA) . Благодаря особенностям платформы, программы написанные на java гарантированно работают на всех операционных системах, для которых существуют реализации виртуальной java машины (Платформенная независимость).

Сборка мусора

В некоторых языках программирования (например таких как C++) контроль за использованием памяти возложен на плечи программиста, что усложняет процесс разработки и приводит к большому количеству ошибок и утечкам в памяти.

В java контроль за использованием памяти возложен на специальный механизм, который называется сборкой мусора (*garbage collector*). Наличие механизма автоматической сборки мусора (более не используемых данных) упрощает процесс разработки и уменьшает количество ошибок при работе с памятью.

1.2. Жизненный цикл, компиляции и исполнения приложения Java

Разработка приложений на java начинается с написания исходного кода. Код хранится в файлах с расширением java. Исходный код преобразуется (компилируется) программой компилятор в байт-код (файлы имеющие расширения class) (рис. 1).



Рисунок 1.

Для выполнения программы, байт-код запускается на виртуальной машине, которая интерпретирует его в команды операционной системы, которая в свою очередь передает команды аппаратной части электронного устройства.

1.3. Инсталляция платформы Java

Шаг 1

Найти и скачать установочный пакет можно по адресу <http://www.oracle.com/technetwork/java/javase/downloads/index.html>.

Выберете версию JDK и операционную систему (обратите внимание на разрядность) на которую будет установлена платформа.

Шаг 2

Запустите установочный пакет (рис. 2).

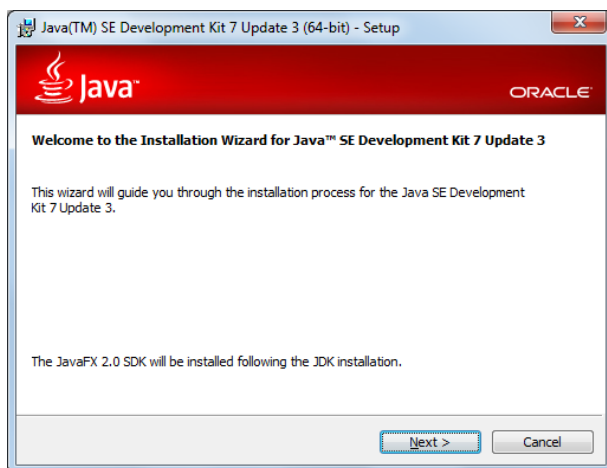


Рисунок 2.

Шаг 3

При необходимости смените путь для установки JDK (рис. 3).

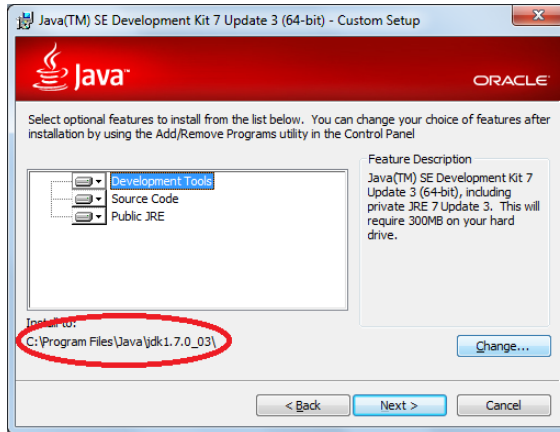


Рисунок 3.

Шаг 4

Установка JRE начнется сразу после установки JDK (рис. 4).

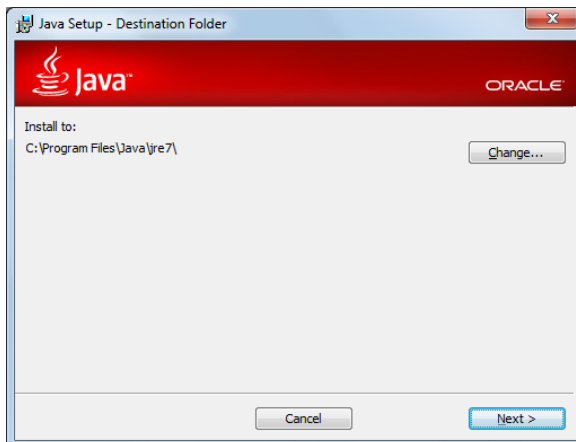


Рисунок 4.

Процесс установки отображается в виде прогресса (*Status*) на экране окна (рис. 5).



Рисунок 5.

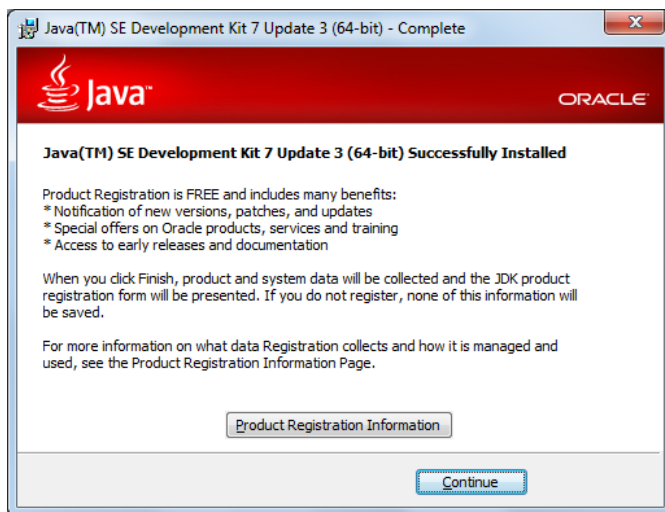


Рисунок 6.

Окончание установки. Есть возможность зарегистрировать установленную JDK на сайте «Oracle» нажав на кнопку «Product Registration Information». Но это не обязательный шаг (рис. 6).

Начиная с версии JSE 1.7 в установочный пакет входит платформа JavaFX и ее установка начнется сразу после установки JRE (рис. 7).

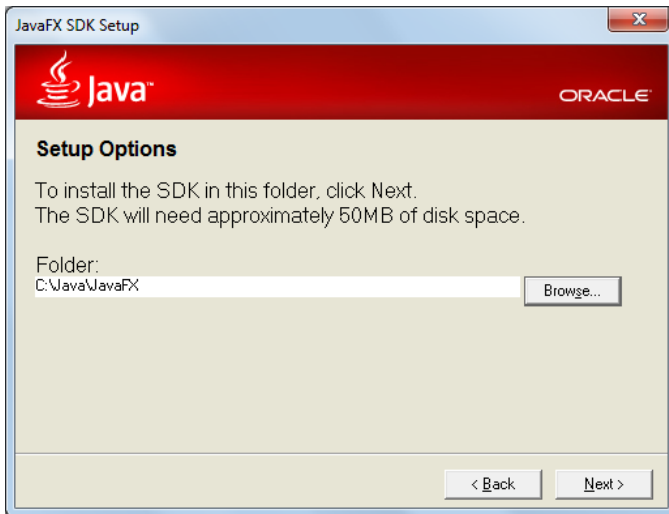


Рисунок 7.

Шаг 5

После установки java платформы необходимо сконфигурировать операционную систему для работы с java.

Для Windows найдите «Компьютер» («Мой компьютер») и выберите его с помощью правой кнопки мыши, в появившемся меню выберите пункт «Свойства».

В появившемся окне выберите пункт «Дополнительные параметры системы» (рис. 8).

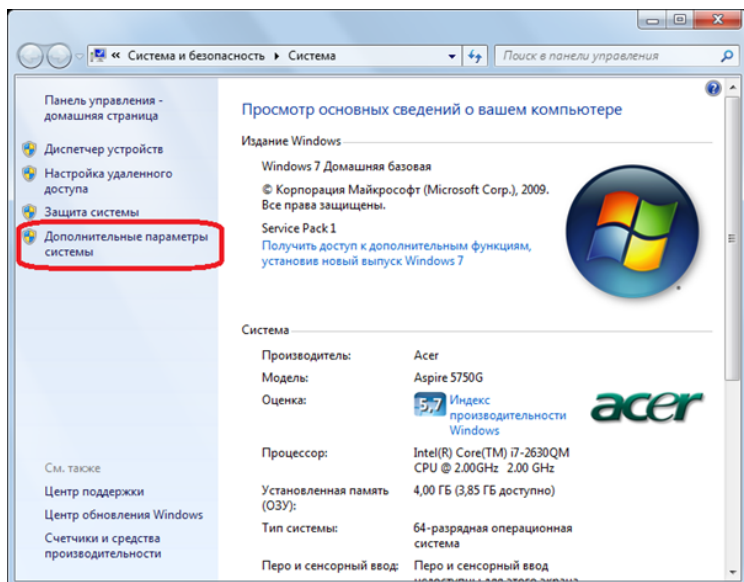


Рисунок 8.

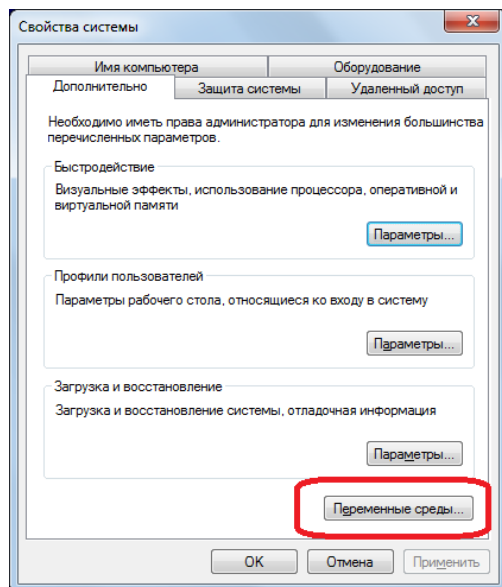


Рисунок 9.

В новом окне необходимо нажать кнопку «Переменные среды» (рис. 9).

Выберите кнопку «Создать» (рис. 10).

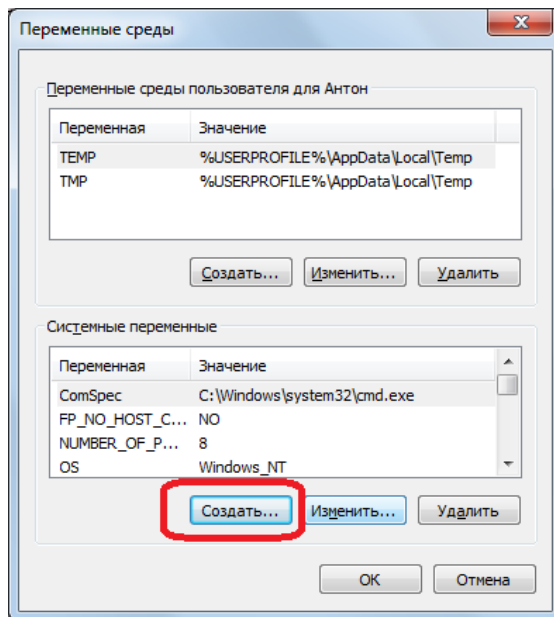


Рисунок 10.

В поле «Имя переменной» внесите значение «JAVA_HOME», в поле «Значение переменной» внесите путь куда была установлена JDK и нажмите «ОК» (рис. 11).

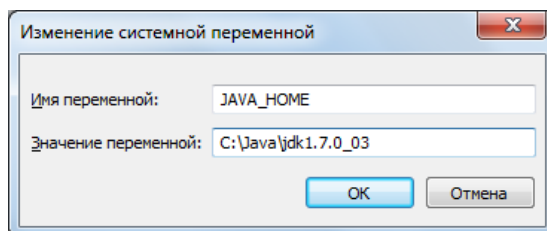


Рисунок 11.

Найдите и выберите в окне «Переменные среды», в поле «Системные переменные» пункт «Path», и нажмите кнопку «Изменить». В поле «Значение переменной» добавьте точку с запятой и путь к директории, где установлены утилиты JDK (директория bin) (рис. 12).

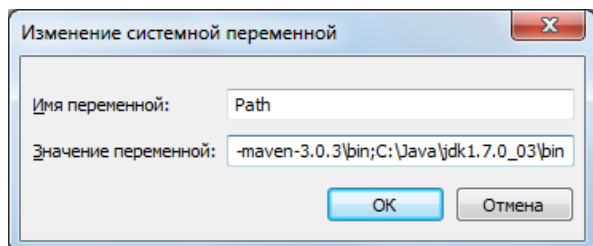


Рисунок 12.

ВНИМАНИЕ!!! Не очищайте и не удаляйте информацию из поля «Значения переменной».

JRE (Java Runtime Environment) — среда выполнения для java приложений. Минимальный набор необходимый запуска java приложений. Содержит виртуальную машину и библиотеки java классов.

JDK (Java Development Kit) — это набор для java разработчика, включающий в себя:

- компилятор (javac);
- стандартные библиотеки классов;
- примеры кода;
- документацию;
- утилиты;
- исполнительную систему (java).

Java — исполняемый файл (находящийся в ...\\Java\\jdk\\bin) запускающий виртуальную машину и передающий байт-код для интерпретации. Файлы с байт-кодом указываются в качестве параметров.

Например: `java.exe -d HelloWorld.class`.

Javac — компилятор исходного кода в байт-код.

Например: `javac.exe HelloWorld.java`.

Jar — утилита создания архива (java ARchive) ZIP формата, в котором упаковываются файлы с байт-кодом и ресурсы программы. Предназначен для объединения модулей программы в цельные логические единицы. Jar файл может быть исполняемым, если содержит файл манифеста.

Javadoc — утилита создания документации на основе исходного кода.

Appletviewer — утилита просмотра приложений типа апплет.

1.4. Пример написания первого приложения

Почти во всех средах разработки программного кода на java, сам процесс компиляции и запуска программы скрыт от пользователя набором различных конфигураций запуска. В результате этого сокрытия многие начинающие разработчики не знают, как скомпилировать или запустить свою программу без IDE, хотя такие знания могут пригодиться при формировании различных скриптов сборки.

Запустить первую программу без IDE можно в три простых шага.

Шаг 1.

Создайте в блокноте файл HelloWorld.java. Заполнить файл кодом из листинга 1.1 и сохранить его.

Листинг 1.1.

```
/**
 * Моя первая программа
 */
public class HelloWorld
{

    /**
     * Точка входа в программу
     */
    public static void main(String[] args)
    {
        System.out.println("Hello world!");
    }

}
```

ВАЖНО!!! *Имя файла и регистр букв в имени файла должен в точности совпадать с именем класса в программе (следующим за ключевым словом class).*

Шаг 2.

Откройте утилиту командной строки. Введите «javac HelloWorld.java» и нажмите на клавиатуре клавишу «Enter». В случае успешной компиляции появится новый файл, имеющий то же имя, но с расширением «class». Это файл с байт-кодом вашей программы (рис. 13).

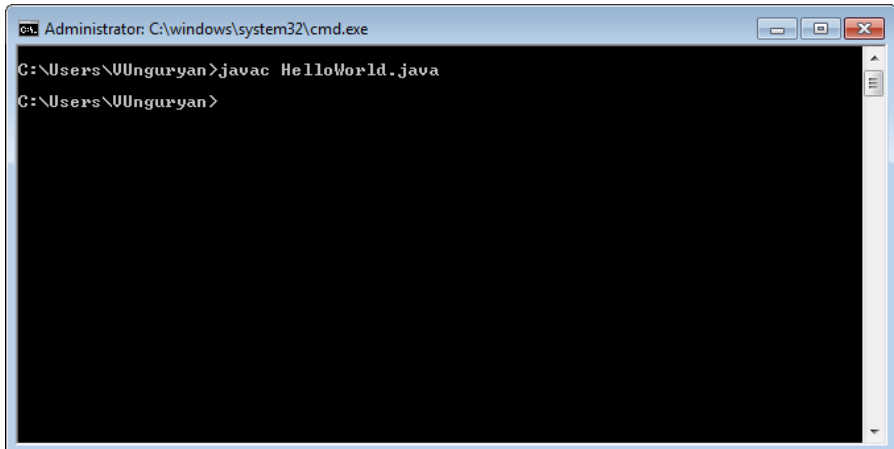


Рисунок 13.

Шаг 3.

Введите в командную строку `java HelloWorld.class` и нажмите «Enter». Программа выполнится, после ее выполнения в окне командной строки будет выведено сообщение: «Hello world!» (рис. 14).

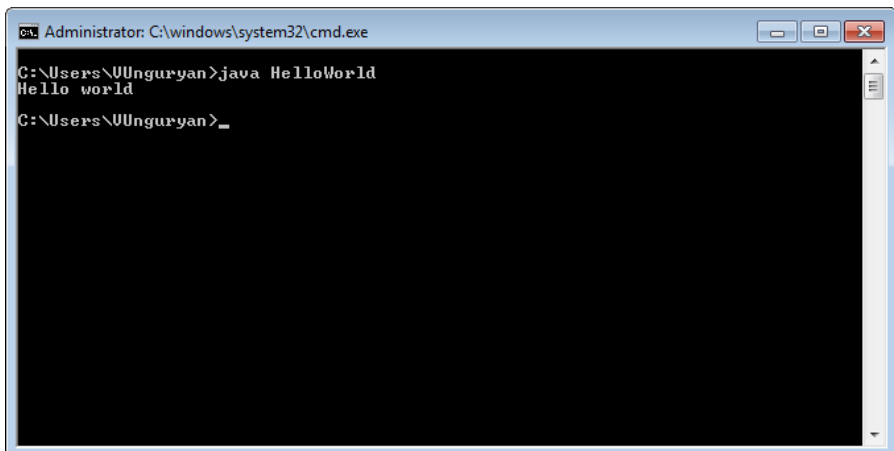


Рисунок 14.

1.5. Обзор существующих IDE

Для упрощения и ускорения процесса разработки программ под java создано большое количество различных интегрированных сред разработки, далее **IDE** (*integrated development environment*).

Все IDE можно разделить на две основные категории, которые существенно влияют на выбор:

- платные;
- бесплатные.

Так же не маловажными параметрами при выборе IDE является время требуемое для первоначальной загрузки, время установки среды, объем занимаемой в ОЗУ и виртуальной памяти.

Лидерами по частоте использования являются: NetBeans, Eclipse, IntelliJ IDEA.

Как правило многие продуктовые компании диктуют программисту, какую IDE ему использовать на проекте, поэтому будет очень хорошо, если Вы будете иметь представление и небольшой опыт в каждой из популярных IDE.



NetBeans — бесплатная свободная среда разработки с открытым исходным кодом. Поддерживает различные языки программирования и разметки. Поддержка большинства современных java технологий. Присутствует интерфейс на русском языке и большое русскоязычное сообщество (рис. 15).

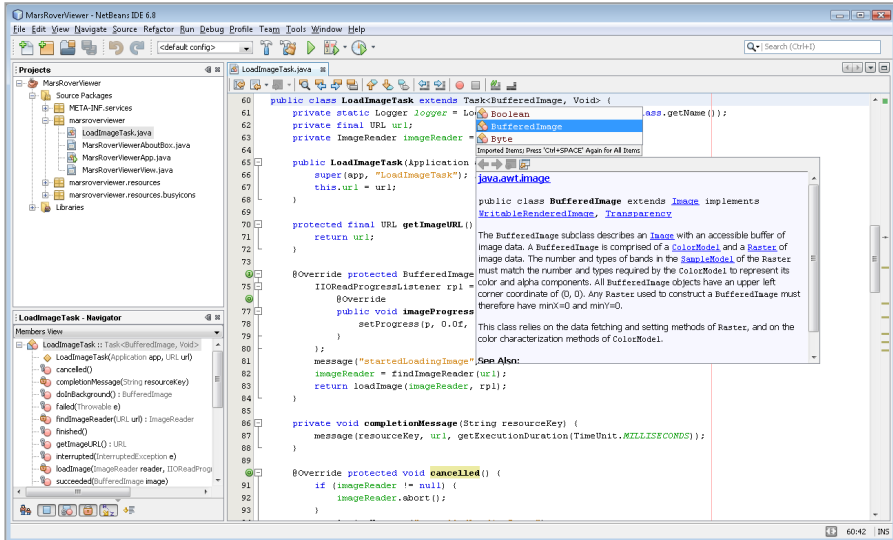


Рисунок 15.

Официальный сайт <https://netbeans.org/>.



IDE Eclipse — бесплатная свободная среда для разработки много-модульных и кроссплатформенных приложений. Имеет открытый исходный код. Развивается и поддерживается некоммерческой организацией «Eclipse Foundation». Изначально среда принадлежала IBM. Представляет собой многослойный конструктор состоящий из сервисной платформы «OSGI» и «SWT/JFace». Расширение функционала осуществляется с помощью плагинов (plugins), которых имеется большое количество (рис. 16).

Официальный сайт <https://eclipse.org/>.

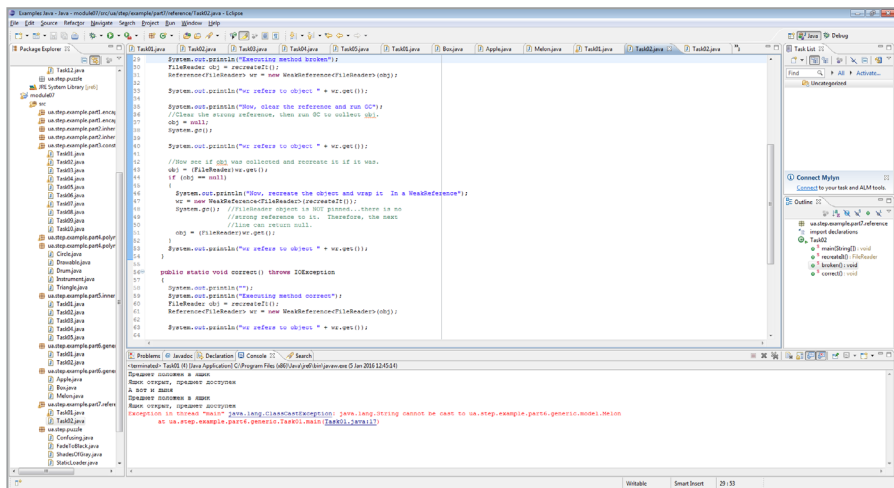


Рисунок 16.



IntelliJ IDEA

IntelliJ IDEA — условно-бесплатная интеллектуальная среда разработки. Разрабатывается компанией «Jet Brains». Отличительной особенностью является понимание контекста и большого количества поддерживаемых технологий и фреймворков прямо из коробки. Имеется бесплатная версия Community (отсутствует поддержка JEE технологий) и платная Ultimate (30-дневный пробный период).

Официальный сайт <https://www.jetbrains.com/idea/>.



Codenvy — бесплатная среда разработки не требующая установки приложения на компьютер. Разработка

ведется в браузере, после загрузки в него необходимого контента (рис. 17).

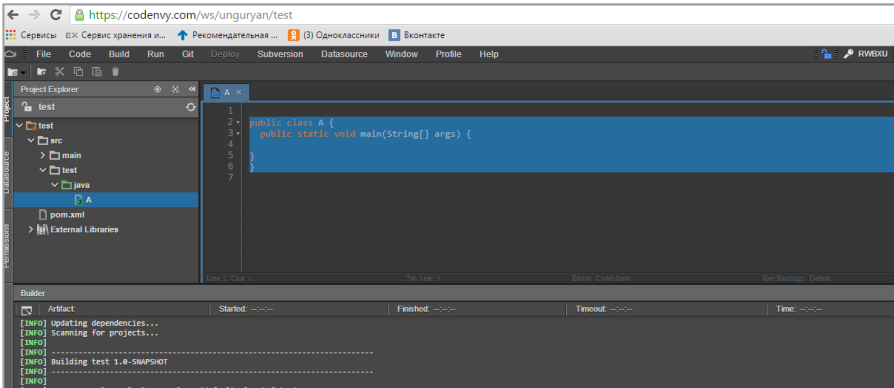


Рисунок 17.

Официальный сайт <https://codenvy.com/>.

Мы будем использовать среду разработки Eclipse, поскольку она бесплатно предоставляет возможности промышленной разработки программ на Java.

Загрузка IDE

Загрузить IDE Eclipse можно бесплатно с официального сайта Eclipse Foundation: <https://www.eclipse.org/downloads/>.

Запустив программу-установщик (файл которой был загружен с сайта), нужно выбрать Eclipse IDE for Java EE Developers, а затем указать папку, в которую будет распакован архив с программой. Рекомендуется использовать папку, путь к которой не содержит пробелов. Например: «c:\users\oiesk\eclipse».

После распаковки IDE готова к работе.

Запуск программы

Запустить на выполнение Eclipse можно двойным щелчком по ярлыку программы на рабочем столе. Также можно запустить файл eclipse.exe, который содержится в целевой папке IDE.

При запуске будет предложено выбрать рабочую область (workspace) (рис. 18).

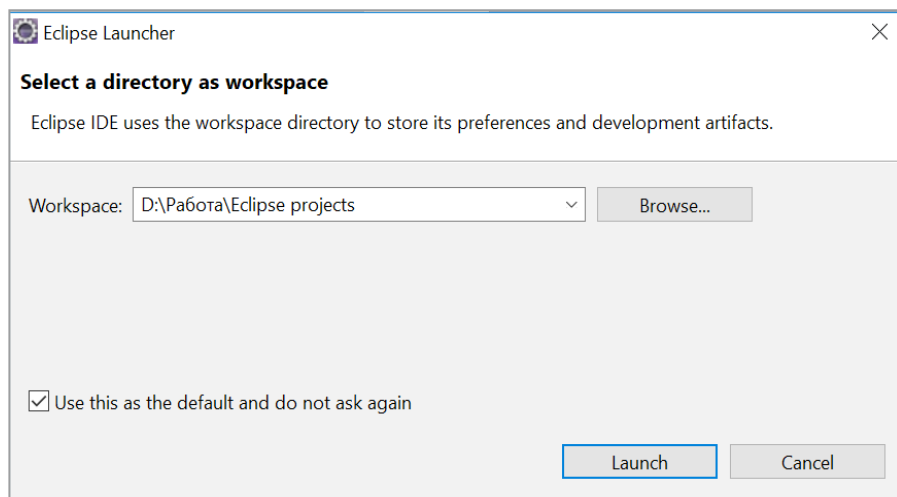


Рисунок 18.

В этой папке будут сохраняться создаваемые проекты и различные настройки среды. Лучше, чтобы это была папка, отличная от той, в которую была распакована программа.

Если Вы не хотите, чтобы при следующих запусках программы это окно возникало снова, установите флажок «Use this as the default and do not ask again».

После запуска появляется страница приветствия (рис. 19).

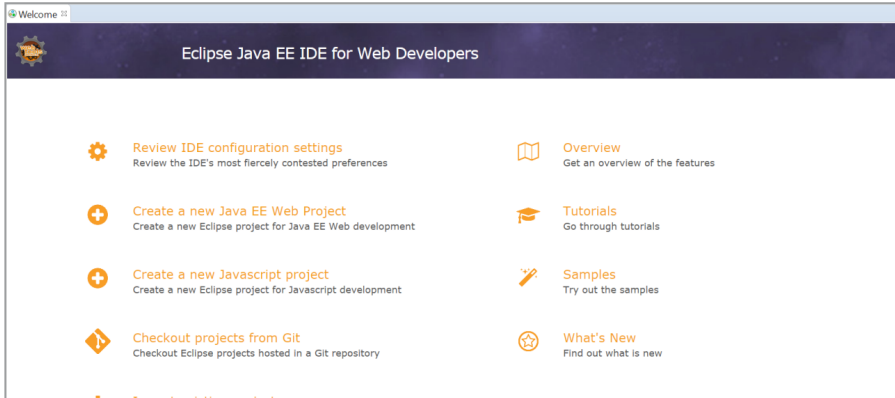


Рисунок 19.

Она содержит ссылки на руководство пользователя, на примеры проектов, новости и т.п. Позволяет быстро создать проект или открыть уже существующий. В этом разделе окно приветствия не будет использоваться, поэтому закроем его.

Создание проекта

Проект в Eclipse представляет собой папку, в которой размещаются все файлы некоторого приложения. По умолчанию создаваемый проект размещается в рабочей области, которая была указана запуске Eclipse. Однако можно создавать или открывать существующий проект и в другой папке (это бывает полезно при переходе на Eclipse после использования других IDE).

Для создания проекта выберите пункт меню «File/New/Project». Появится диалоговое окно для выбора типа проекта. Следует выбрать «Java Project». После нажатия кнопки «Next» открывается окно создания проекта (рис. 20).

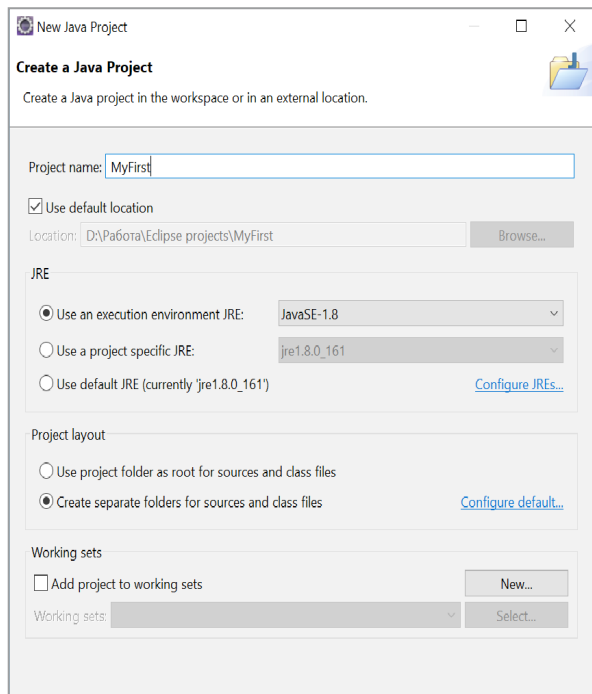


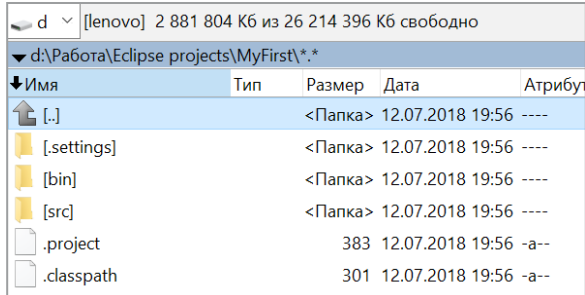
Рисунок 20.

Зададим имя проекта MyFirst и поставим флажок «Use default location», чтобы файлы проекта размещались в заданной рабочей области.

Затем идет раздел выбора версии JRE (разумеется, стоит выбрать самую свежую версию из установленных на данном ПК). Переключатель «Create separate folders for sources and class files» следует оставить активированным, чтобы файлы с исходными кодами классов и файлы с байт-кодом помещались в отдельных папках.

Для завершения диалога и создания проекта нужно нажать кнопку «Finish». После этого возникает диалоговое окно, в котором система уточняет, согласны ли Вы

загрузить перспективу, связанную с Java. Согласитесь с этим, нажав кнопку «yes» (понятие перспективы обсудим в следующем пункте). В итоге в рабочей области будет создана папка проекта «MyFirst» (рис. 21).



Имя	Тип	Размер	Дата	Атрибут
[.]	<Папка>		12.07.2018 19:56	----
[.settings]	<Папка>		12.07.2018 19:56	----
[bin]	<Папка>		12.07.2018 19:56	----
[src]	<Папка>		12.07.2018 19:56	----
.project		383	12.07.2018 19:56	-a--
.classpath		301	12.07.2018 19:56	-a--

Рисунок 21.

Папка проекта имеет определенную структуру. Она содержит подпапку `src` — для хранения исходного кода классов (файлы с расширением `.java`), `bin` — для хранения байт-кода (с расширением `.class`) и `.settings` — для хранения настроек (файл с расширением `.prefs`).

Интерфейс программы

Вид экрана зависит от настроек программы, которые программист выбирает под свои нужды. Совокупность таких настроек называется **перспективой** (*perspective*). Другие переводы: ракурс, профиль, проекция.

В понятие перспективы входят отображаемые панели просмотра (*views*) и панели инструментов, а также их расположение на экране. Также в перспективу входит выбор редактора кода программы (обычный текстовый редактор, редактор для Java, редактор для C++ и т.д.). Перспективы можно сохранять и загружать по мере необходимости.

При загрузке IDE Eclipse имеется набор стандартных перспектив. Во время работы они меняются автоматически соответственно выполняемым действиям.

На рисунке показан стандартный вид экрана после создания проекта на Java (рис. 22).

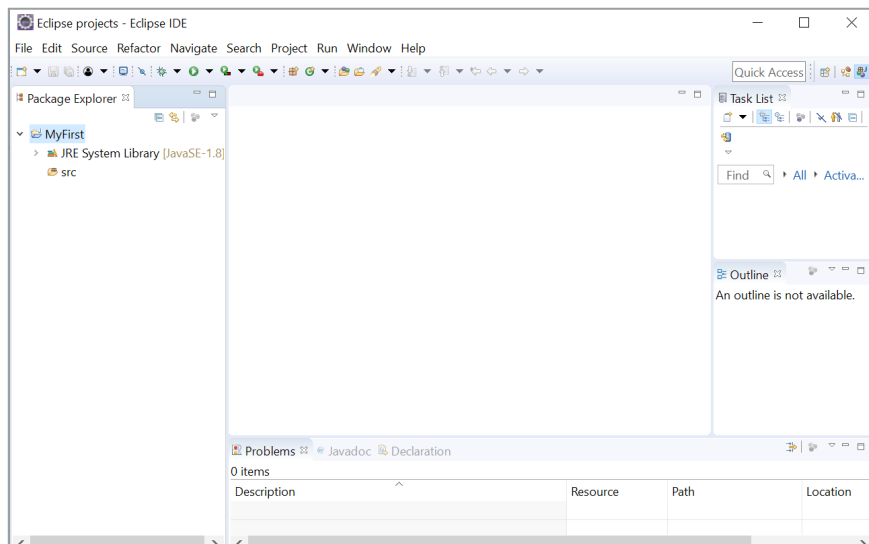


Рисунок 22.

В верхней части экрана расположено главное меню и панели инструментов. Красным кружком выделена кнопка выбора перспективы (). Нажав ее, можно увидеть список встроенных перспектив.

В центре рабочего пространства — область редактора кода (именно здесь вводится код программы).

Слева — панель обозревателя пакетов «Package Explorer». В ней показана иерархия пакетов и классов проекта.

Справа внизу — панель «Outline». В ней будут показаны поля и методы класса, открытого в редакторе.

Справа вверху — список задач «Task List». Это окно позволяет организовать работу программиста над проектом. В нее можно записывать задачи типа «Добавить кнопку такую-то в модуле таком-то», «Исправить ошибку в методе таком-то». И даже задачи личного характера. Например, «Записаться к дантисту». Для задач можно назначить сроки исполнения и категорию.

Внизу окно с несколькими вкладками, позволяющими контролировать компиляцию и выполнение программы. Так, вкладка «Problems» предназначена для вывода ошибок компилятора. Вкладка «Console» показывает содержимое консоли в результате выполнения программы. Вкладка «Javadoc» отображает комментарии к выделенным элементам в тексте программы. А на вкладке «Declaration» можно увидеть строку кода с объявлением выделенного элемента программы (например, если нужно вспомнить с каким типом была объявлена переменная).

Для управления элементами интерфейса экрана (вывода/удаления панелей просмотра, загрузки перспективы или смены редактора) служит пункт главного меню «Window».

Для изменения параметров интерфейса Eclipse нужно использовать команду: «Window/Preferences».

Например, чтобы изменить размер или начертание шрифта в редакторе кода, задайте: «Window/Preferences/General/Appearance/Colors and Fonts/Basic/Text Font/Edit».

Создание класса

В панели «Package Explorer» выберите папку src и вызовите контекстное меню правой кнопкой мыши. В меню нужно выбрать «New/Class». Появится окно диалога «New Java Class» (рис. 23).

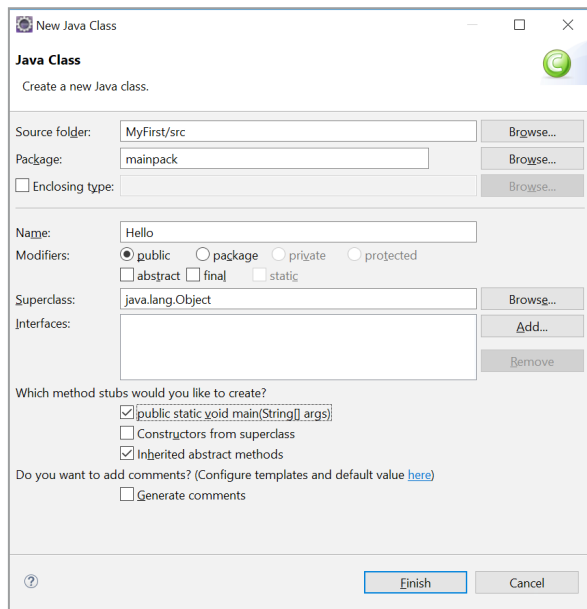


Рисунок 23.

В этом окне автоматически прописывается папка, в которую будет помещен созданный java-файл (MyFirst/src). Менять ее не будем.

Можно указать имя пакета (например, mainpack). Если этого не сделать, то пакет будет задан по умолчанию (*default*). Пакет соответствует папке на диске. Использование пакетов позволяет упорядочить классы проекта в виде четкой иерархической структуры.

В поле «Name» задается имя класса, которое совпадает с именем соответствующего файла (например, Hello). Обратите внимание, что это имя обязательно должно начинаться с прописной буквы.

Далее можно выбрать модификатор доступа для класса, суперкласс и интерфейсы, которые он должен

реализовать. Пока не будем здесь ничего задавать, оставим по умолчанию.

Чтобы в класс был автоматически помещен метод `main()`, установите флажок «`public static void main(String[] args)`».

Нажатие кнопки «Finish» завершает диалог. В окне редактора появляется вкладка с заготовкой кода класса. Изменится и содержимое панели «Package Explorer», в которой появится пакет «mainpack» и файл `Hello.java` (рис. 24).

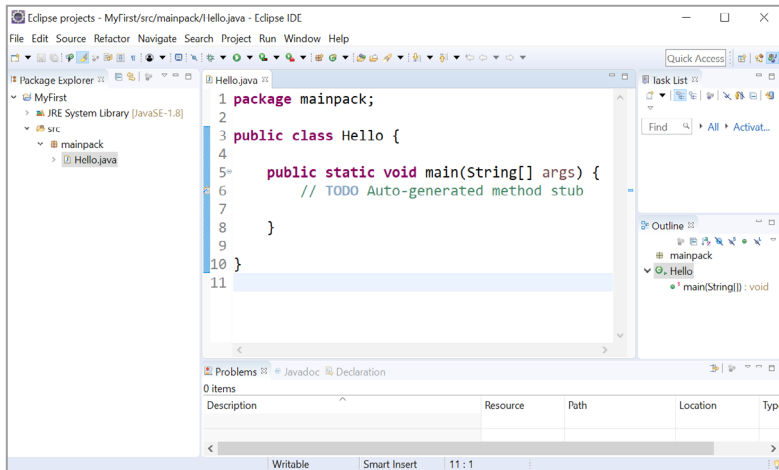


Рисунок 24.

Если проанализировать папку проекта после создания класса, то в папке `src` можно обнаружить папку `mainpack` и в ней файл `Hello.java`.

Ввод кода программы, компиляция и выполнение

Введем код программы, приведенный в листинге 1. Обратите внимание, что редактор кода выдает подсказки во время ввода программы (рис. 25).

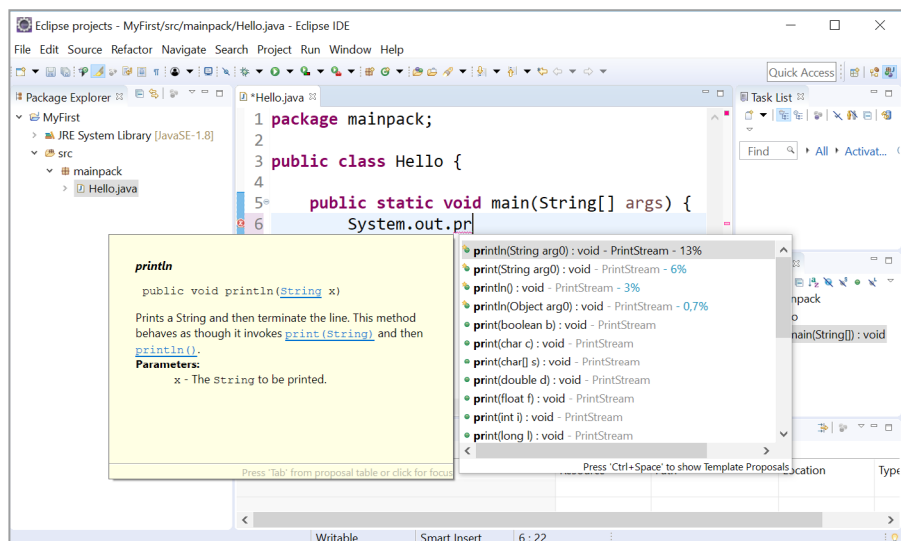


Рисунок 25.

Содержание этих всплывающих окон зависит от контекста. Например, в таком окне может содержаться список методов, которые можно вызвать от имени данного объекта. Можно выбирать из предлагаемого списка, и код будет дописан автоматически.

Для удобства ввода можно раскрыть/свернуть область редактора клавишами «Ctrl+M».

Редактор использует выделение цветом для обозначения того, как компилятор будет воспринимать то или иное слово программы. Так, ключевые слова выделяются фиолетовым, строковые константы — голубым, а комментарии — зеленым.

Ошибки редактор подчеркивает розовой волнистой линией, а во всплывающем окне к этой ошибке даются рекомендации по ее устранению.

Для выполнения компиляции и последующего запуска программы на исполнение можно нажать кнопку на панели инструментов (либо выбрать из контекстного меню редактора «Run As/1 Java application», либо нажать горячие клавиши «Ctrl+F11», либо воспользоваться пунктом «Run» главного меню).

При первом запуске компилятора IDE предложит сохранить текст программы перед запуском (рис. 26):

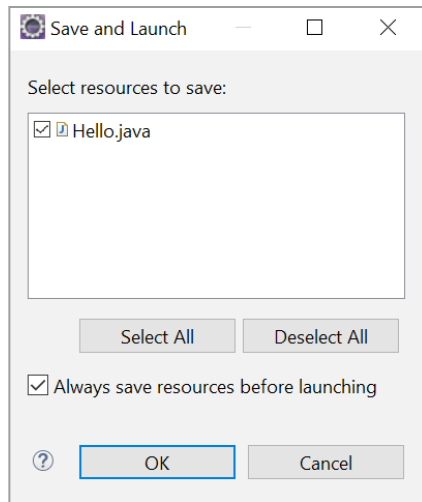


Рисунок 26.

Поставьте флажок «Always save resources before launching», чтобы изменения в коде автоматически сохранялись при каждом запуске.

Результат выполнения первой программы — вывод строки «Здравствуй, мир!» в окне консоли (появляется внизу рабочего пространства программы).

Если необходимо создать новый проект в той же рабочей области, то нужно вызвать контекстное меню

в окне «Package Explore» и задать «New/Java Project». Задание параметров нового проекта и добавление классов в этот проект выполняются аналогично тому, как это было описано выше.

Основные горячие клавиши Eclipse

- **Ctrl+M** — максимизировать/минимизировать любое окно, в котором находится курсор.
- **Ctrl+F7** — переключение между окнами.
- **Ctrl+Space** — вывод вариантов автозавершения. В процессе ввода кода программы всплывающие окна подсказок появляются не всегда. Но если нажать сочетание «Ctrl+Space», появляется всплывающее окно с возможными вариантами завершения вводимого кода. Выбор одного из них позволяет автоматически дописать фрагмент. Этот способ хорош еще и тем, что позволяет не держать в голове список всех методов используемого класса и подробности их интерфейсов. «Eclipse» дает описание выбранного из списка метода в дополнительном всплывающем окне.
- **Ctrl+1** — автоматическая генерация кода. При нажатии «Ctrl+1» во всплывающем окне «Eclipse» выводит список действий, возможных в данном контексте (в зависимости от ситуации это может быть и создание нового класса, и описание переменной, и импорт пакета с библиотекой классов, и много другое). Удобно использовать эту функцию для исправления ошибок. Для этого нужно поставить курсор на ошибку и нажать «Ctrl+1». «Eclipse» предложит варианты ее исправления.

- **Ctrl+Shift+L** — вывод списка всех горячих клавиш.
- **Alt+Up/Down Arrow** — перемещение текущей строки или выделенного фрагмента вверх/вниз по коду.
- **Ctrl+Shift+R** — открывает любой ресурс (это может быть java-код, xml-файл, текстовый файл и т.д.)
- **Ctrl+Shift+T** — открывает любой класс. В поле ввода можно задать первые буквы имени класса или использовать шаблон имени. Также есть возможность ввода только заглавных букв имени класса (например, «AL» вместо «ArrayList»). «Eclipse» предложит подходящие варианты, среди которых выбирается нужный.
- **Ctrl+/ **— закомментировать/раскомментировать выделенный фрагмент кода.****
- **F4** — вывод окна иерархии типов (*Type Hierarchy*). При нажатии этой клавиши должно быть выделено имя какого-либо класса в программе. В появившемся окне можно посмотреть иерархию предков класса, а также список его полей и методов.
- **F3** — вывод окна описания класса [Declaration](#) в справочной системе. Аналогично должно быть выделено имя класса в программе.
- **Ctrl+L** — переход к строке кода с определенным номером.
- **Ctrl+Q** — переход к последней строке, которая редактировалась.
- **Ctrl+o** — открывает всплывающее окно со списком доступных для редактирования классов и методов. Удобно для быстрого перехода к нужному методу.

- Использование шаблонов автозавершения в сочетании с клавишами «Ctrl+Space». Например, «sysout + (Ctrl + Space)» выводит `System.out.println();` `fore+(CTRL+Space)` выводит шаблон для цикла `foreach` `for(String string;args){}`. Полный список всех предустановленных шаблонов можно увидеть, задав команду «Window/Preferences/Java/Editor/Templates». В этом же окне можно создать и свой собственный шаблон. Например, на рисунке показан пример создания шаблона для комбинации `psvm` (рис. 27).

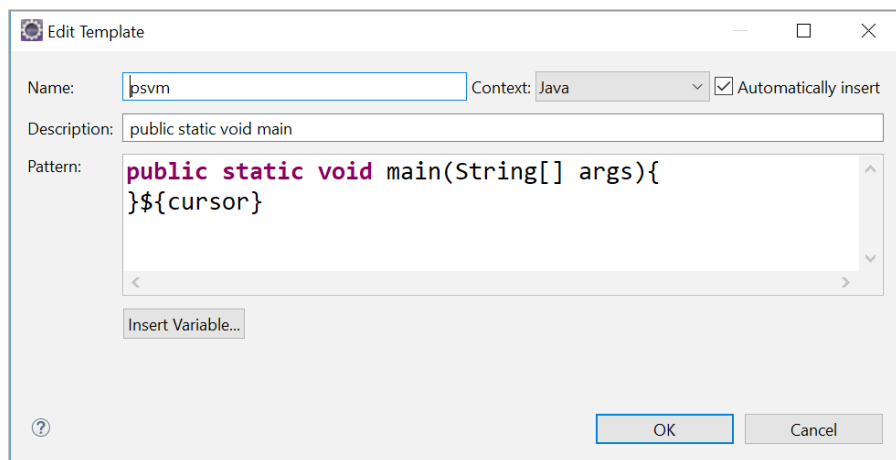


Рисунок 27.

2. Базовые конструкции

2.1. Понятие о строгой типизации

В java при объявлении переменной всегда необходимо указывать ее тип. После объявления переменной она не может изменить свой тип на протяжении всей видимости переменной в коде. Строгая типизация позволяет обнаруживать ошибки в коде на этапе компиляции программы.

2.2. Типы данных

В языке java есть две основные категории делящих типы переменных на примитивные и ссылочные.

Примитивные типы данных (базовые типы) — это типы данных присутствующие в синтаксисе языка. Для использования примитивных типов нет необходимости создавать свои классы или использовать какие-либо библиотеки.

Ссылочный тип (объектный тип) — это тип который в качестве значения содержит ссылку на объект.

Значения переменных ссылочного типа хранятся в динамической памяти (куче). А в стеке располагается только ссылка на соответствующий объект (можно на начальном этапе представлять себе ссылку как автоматически разыменовываемый указатель). А значения примитивных типов хранятся непосредственно в стековой памяти. Отличия ссылочных типов от примитивных иллюстрирует следующий рисунок (рис. 28).

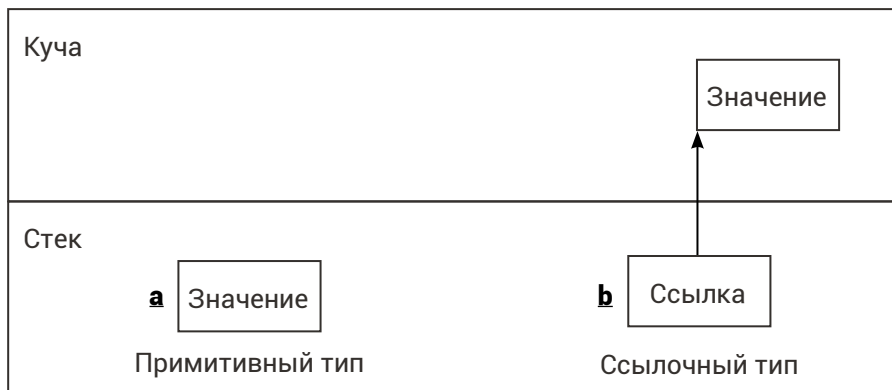


Рисунок 28.

Примитивные типы данных делятся на:

Целочисленные примитивные типы данных могут хранить в себе только целые числа (без дробной части) из диапазона заданного размерностью типа.

Таблица целочисленных типов

Наименование	Разрядность	Диапазон значений
<code>byte</code>	8 бит (1 байт)	от -128 до 127
<code>short</code>	16 бит (2 байта)	от -32 768 до 32 767
<code>int</code>	32 бита (4 байта)	от -2147483648 до 2147483647 или от -2^{31} до 2^{31}
<code>long</code>	64 бита (8 байт)	от -2^{63} до 2^{63}

Дробные типы данных предназначены для хранения значений дроби. Дробный тип данных может быть представлен в виде десятичной дроби. В качестве разделителя целой и дробной части используется точка. Например 1.5 — это одна целая и пять десятых. Так же могут быть использованы научная или экспоненциальная форма записи.

Таблица дробных типов

Наименование	Разрядность	Диапазон значений
<code>float</code>	32 бита (4 байта)	от $-3.4e-038$ до $3.4e+038$
<code>double</code>	64 бита (8 байт)	от $-1.7e-308$ до $1.7e+308$

Особенности специальных значений
с плавающей точкой

Выражения	Результат
<code>Math.sqrt(-1.0)</code>	NaN
<code>0.0 / 0.0</code>	NaN
<code>1.0 / 0.0</code>	Infinity
<code>-1.0 / 0.0</code>	-Infinity
<code>NaN==NaN</code>	false
<code>Infinity==Infinity</code>	false

Символьный тип может хранить в себе только целые числа, которые интерпретируются, как коды символов из таблицы на основе юникода (*Unicode*). Юникод — это стандарт кодирования символов, в котором каждому символу соответствует определенный код. В юникоде представлены все возможные знаки различных народов мира. Каждый символ кодируется двумя байтами. Если отсутствует возможность набрать в коде символ с клавиатуры он может быть заменен на запись символа в виде юникода. Например: `'\u0056'`.

Наименование	Разрядность	Диапазон значений
<code>char</code>	16 бит (2 байта)	от 0 до 65536

Логический тип используется для хранения значений полученных в результате вычисления логических

выражений или может задаваться логическими литералами: где `true` — обозначает истину, `false` — обозначает ложь. В стандартной реализации «Sun JVM» для хранения значений типа `boolean` используется 32 бита, а в случае массива `boolean` происходит оптимизация до 8 бит. Для хранения большого количества значений типа `boolean` рекомендуется использовать класс [BitSet](#).

Наименование	Разрядность	Диапазон значений
<code>boolean</code>	—	<code>true</code> или <code>false</code>

Преобразование типов

Java является языком со строгой (статической) типизацией и всегда контролирует работу с данными определенного типа, но не смотря на это java дает программисту возможность копировать значения одного типа в другой.

В случае если размерность типа копируемого значения меньше размерности типа в который копируется, автоматически происходит *неявное преобразование* типа. Неявное преобразование не требует специального синтаксиса.

Пример:

```
short i = 3;
int j = i;
long n = 5; // число 5 это литерал типа int
```

Явное преобразование типов может происходить и в выражениях. Все операнды не явно преобразуются в тип, который имеет наибольшую разрядность.

Пример:

```
byte a = 37;
short b = 12;
char c = 'a';
int sum = a + b + c; // тип всех операндов повышается
                     // до int
```

Таблица не явных преобразований

Исходный тип	Типы в которые можно преобразовать неявно	Потеря точности
byte	short, int, long	Нет
short	int, long	Нет
int	long	Нет
int	float, double	Да
char	int	Нет
long	float, double	Да
float	double	Нет

Пример неявного преобразования с потерей точности:

```
int big = 1234567890;
float f = bigNumber;
System.out.printf("%f", f);
```

Результат: 1234567936.000000.

Если есть необходимость скопировать значение типа который имеет большую разрядность в тип с меньшей разрядностью, то необходимо использовать *явное преобразование* типов.

Синтаксис явного преобразования

```
<тип> i = (<тип>) значение исходного типа;
```

Пример:

```
int i = (int) 7L;
float m = (float) 1.5; // 1.5 является литералом тип
                        // которого double
int j = 12;
byte b = (byte) j;
```

Пример ошибки неявного приведения типа:

```
byte b = 55;
b = b * 2; // ошибка компиляции, так как 2 это
           // литерал типа int и результатом
           // выражения тоже будет тип int
```

При явном и не явном преобразовании типов может произойти переполнение (*overflow*) или потеря значений (*underflow*). Java никак не контролирует эти процессы и не выводит ошибки при компиляции или выполнении программы.

Пример overflow при не явном приведении:

```
byte bMax = 127;
bMax++;
System.out.println(bMax);
```

Результат: -128.

Пример overflow при явном приведении:

```
short maxValue = 256;
byte bb = (byte) maxValue;
System.out.println(bb);
```

Результат: 0.

Пример underflow:

```
double d1 = 0.3333333333333333;
// потеря чисел после точки начиная с 8 знака
float f1 = (float) d1;
System.out.println(f1);
```

Результат: 0.33333334.

Пример underflow:

```
float f3 = 3.64f;
int i3 = (int) f3; // дробная часть отбрасывается
System.out.println(i3);
```

Результат: 3.

ВАЖНО!!! При явном преобразовании типов, контроль за переполнением значения или потери точности полностью ложится на плечи программиста.

Типы-обертки (wrapper classes)

Для всех примитивных типов в java существуют классы являющимися их обертками (оболочками) предназначенные для работы с значениями примитивных типов как с объектами (ссылочными типами).

Так как Java является объектно-ориентированным языком, то все данные, следуя парадигме ООП, должны инкапсулироваться в объектах. Но использование объектов для математических вычислений существенно снижает быстродействие, усложняет код и занимает больший объем памяти. Поэтому в java кроме ссылочных типов

были введены примитивные типы, которые являются небольшим исключением из объектной модели языка. Классы обертки возвращают работу с примитивными типами к объектной модели.

Пример сложения с использованием объектов:

```
Integer sum = new Integer(2) + new Integer(2);
```

Пример сложения с примитивными типами:

```
int sum = 2 + 2;
```

Все классы обертки являются неизменяемыми (нельзя изменить значение хранящееся внутри объекта класса-обертки), так же нельзя наследоваться от классов-оберток. Эти правила связаны с логическим предназначением классов-оберток, которые оборачивают числовые значения.

Примитивный тип	Тип-оболочка
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Все целочисленные типы-обертки наследуются от абстрактного класса **Number**.

Значения передаются в класс-обертку с помощью конструктора, в виде литерала или переменной примитивного типа или в качестве строки.

Пример:

```
Integer i = new Integer(3);
Integer i1 = new Integer("123");
Number n = new Short(4);
```

Классы-обертки имеют статические и не статические методы.

Статические методы

Методы вида **parseXXX** преобразуют строку в значение примитивного типа.

Примеры:

```
int i1 = Integer.parseInt("123");
boolean b1 = Boolean.parseBoolean("TRUE");
```

Метод **valueOf** преобразует строку или число в класс-обертку.

Примеры:

```
Integer i2 = Integer.valueOf("5");
Integer i3 = Integer.valueOf(7);
Float f = Float.valueOf(1.3f);
```

Метод **toBinaryString** есть только у классов-оберток целых типов преобразует число в строку в бинарном представлении.

Пример:

```
String binaryString = Integer.toBinaryString(21);
System.out.println(binaryString);
```

Результат: 10101.

Метод `toHexString` преобразует число в строку в шестнадцатеричном представлении.

Пример:

```
String hexString = Float.toHexString(1.5f);
System.out.println(hexString);
```

Результат: 0x1.8p0.

Пример:

```
String hexString = Integer.toHexString(255);
System.out.println(hexString);
```

Результат: ff.

Класс-обертка `Character` имеет множество различных методов присущих только ему. Большинство из них простые и что они делают понятно из названия методов.

Пример:

```
// проверяет является ли символ цифрой
boolean isDigit1 = Character.isDigit('3');
// истина если цифра в верхнем регистре
boolean isUpper = Character.isUpperCase('b');
```

СОВЕТ. *Старайтесь всегда используйте примитивные типы, а обертки используйте только там, где невозможно использовать примитивные типы.*

Автоматическая упаковка и распаковка

Автоматическая упаковка (*autoboxing*) — это механизм не явного создания (без использования опера-

тора `new`) объекта класса-обертки соответствующего примитивного типа.

Пример:

```
Integer i = 5;  
Boolean b = true;
```

Автоматическая распаковка (*unboxing*) — это механизм неявного преобразования объекта типа-обертки в примитивный тип соответствующего типа.

Пример:

```
int i = new Integer(5);
```

Автоматическая упаковка и распаковка может происходить при присвоении или при передаче в метод.

Пример авто-упаковки:

```
public static void main(String[] args)  
{  
    test(3); // литерал 3 упаковывается в объект  
             // класса-обертки  
}  
  
public static void test(Integer value)  
{  
    System.out.println(value);  
}
```

Автоматическая упаковка и распаковка упрощают код, но могут привести к ряду ошибок, которые можно выявить только на этапе выполнения программы.

Пример:

```
Integer i = null;
int j = i; // возникнет исключение NullPointerException
```

Пример:

```
Number n = 555;
byte b = (Byte) n; // возникнет исключение
                  // ClassCastException
```

Пример:

```
static void inc(List<Integer> list, int v)
{
    for (Integer i:list) {
        // здесь не происходит изменение значения
        // переменной в списке
        i +=v;
    }
}
```

Примечание. *Autoboxing* и *unboxing* появились в java с версии *JDK 1.5*.

2.3. Комментарии

Язык Java позволяет вставлять в исходный код различные комментарии, замечания и пояснения, которые не подвергаются компиляции и не попадают в байт-код.

// — строчный комментарий, все что находится после двойных обратных черт до конца строки не будут скомпилировано.

Пример:

```
int i = 5; // int i = 3;
```

Блочный комментарий позволяет выделить часть исходного кода, которая будет исключена из компиляции.

`/*` — начало блочного комментария,

`*/` — конец блочного комментария.

Пример:

```
/*
    Пример блочного комментария
    int i =5;
*/
i = 3; // ошибка компиляции, так как переменная i
        // не объявлена в коде, а находится внутри
        // блока комментариев.
```

Javadoc — особый вид комментариев предназначенный для описания назначения классов, методов и полей классов. В комплект JDK входит утилита, которая собирает в документ (html) все комментарии имеющие тип javadoc по всем файлам исходного кода проекта и на их основе создает html документ API (*application programming interface*).

Пример:

```
/**
 * Описание назначения класса
 * @see Config (вместе с этим смотри класс Config)
 * @author VUnguryan (автор класса)
 * @since 1.2.1 (версия класса)
 */
```

```
public class Controller
{
}
```

[Пример API для java классов.](#)

2.4. Переменные

Переменная — это именованная область памяти, в которую может быть записано или перезаписано и откуда может быть прочитано значение определенного типа.

Тип переменной и ее имя (идентификатор) задаются в момент объявления переменной в программе.

Синтаксис объявления переменной:

```
тип идентификатор;
```

Пример:

```
int a;
byte b1;
boolean $_$;
```

Синтаксис языка позволяет объявить сразу несколько переменных одного типа в одной строке, но так не рекомендуется делать по *Java Code Convention*.

Пример:

```
float x, y, z;
```

Имя переменной:

- может содержать все символы латинского алфавита, цифры, знак \$ и знак _;

- не может начинаться с цифры;
- не может совпадать с [ключевыми словами java](#).

Примеры правильного именования переменных:

```
long startTime;  
int x;  
boolean flag;
```

Примеры неправильного именования переменных:

```
int lside;  
char goto;  
long st@rt;
```

***Примечание.** Использование символов \$ и _ в имени переменных считается плохой практикой.*

Переменные принято именовать согласно рекомендациям документа [Java Code Conventions](#).

Область видимости переменных

Переменная может быть объявлена в любом месте кода до ее использования. Переменная доступна от точки ее объявления до конца блока.

Блок — это любой фрагмент кода, заключенный в фигурные скобки. Например, тело метода является блоком. Поэтому любая переменная, объявленная внутри метода, недоступна за его пределами.

Если в цикле **for** объявляется переменная-параметр цикла, то за пределами цикла эта переменная будет недоступна.

Пример:

```
public class Runner {

    public static void main(String args[])
    {
        {
            // System.out.println(x);
            // переменная x еще невидима
            int x = 12;
            System.out.println(x); // переменная x
                                   // доступна
        }
        // System.out.println(x);
        // переменная x уже невидима
    }
}
```

Еще пример:

```
public class Runner {
    public static void main(String args[])
    {
        for(int i=1;i<=3;i++){
            System.out.println("tik");
        }
        // System.out.println(i);
        // переменная i уже невидима
    }
}
```

В Java нельзя объявить во вложенном блоке переменную, имя которой совпадает с именем переменной во внешнем блоке (в отличие от языка C++).

Пример:

```
public class Runner{
    public static void main(String args[])
    {
        int x=12;
        {
            // int x=18; //ошибка
            System.out.println(x);
        }
    }
}
```

Переменные, объявленные внутри метода, называются *локальными переменными* и перед непосредственным использованием требуют явной инициализации.

Переменная, объявленная внутри тела класса (вне любого метода) называется полем класса. В случае отсутствия инициализации поле класса принимает значение по умолчанию (в зависимости от типа).

Пример:

```
class A
{
    static int x; // поле класса
    public static void main(String[] args) {
        int y; // локальная переменная
        System.out.println(y); // результат ошибка
                                // компиляции с
                                // сообщением1
        System.out.println(x); // результат 0
    }
}
```

¹ The local variable y may not have been initialized

Поле класса доступно внутри любого метода этого класса. При этом объявление поля класса может быть расположено в коде после его использования в методе. Если в методе или в блоке объявлена локальная переменная с тем же именем, что и поле класса, то локальная переменная имеет приоритет («заслоняет собой» поле класса).

Пример:

```
public class Runner {  
    static int x=15;  
    static int y=22;  
    public static void main(String args[])  
    {  
        int x = 12;  
        System.out.println(x); // обращение к локальной  
        System.out.println(y); // обращение к полю  
                                // класса  
        System.out.println(z); // обращение к полю  
                                // класса  
    }  
    static int z=25;  
}
```

Вывод: 12 22 25.

Подробнее область видимости полей класса будет обсуждаться в следующем модуле при знакомстве с модификаторами доступа.

2.5. Константы

Константа — это переменная которая должна быть инициализирована в месте объявления (или в конструкторе класса) и не может изменять своего значения на

протяжении видимости этой переменной в коде. Для обозначения константы используется ключевое слова **final**.

Пример:

```
final int const = 5;
const = 3; // ошибка компиляции
final int const1; // ошибка компиляции
```

Под понятие константы подходит понятие литерала.

Литерал — это явно заданное в коде значение определенного типа. Для задание литералов в коде программы используется специальный синтаксис, см таблицу.

Тип литерала	Тип данных	Спец-символы	Пример
Целочисленный	int long	x l, L	3, 03, 0x3 3l, 3L
Дробный	float double	f, F, e d, D, e	1.5f, 1.5F, 1.5e-1f 1.5, 1.5d, 1.5D, 1.5e-1
Булевый	boolean	true, false	true, false
Символьный	char		'a', '\u0041'
Ссылочный	Все ссылочные	null	null

Примечание. Начиная с версии JDK 7.0 java позволяет разделять разряды числовых литералов символом подчеркивания. Например: 100_000_000. Не рекомендуется использовать маленькую букву *l* для обозначения литерала типа long, так как возможно неверное толкование (похоже на цифру 1)

2.11. Ввод-вывод в консольной программе

Для вывода данных на консоль необходимо использовать класс `System` и его статическую переменную `out`.

Примеры:

```
System.out.println("text"); // вывод текста с переходом
                             // на новую строку
System.out.println(5); // вывод литерала с переходом
                       // на новую строку

int i = 6;
// вывод значения переменной
System.out.println(i);
// вывод текста без перехода на новую строку
System.out.print("text");
// форматированный вывод переменных
System.out.printf("x = %f", 0.5f);
```

Для ввода данных с консоли необходимо использовать класс `java.util.Scanner`. Класс `Scanner` имеет ряд методов, позволяющих работать с потоком ввода.

Класс `Scanner` представляет из себя поток данных, получаемых из источника.

Когда используется класса из библиотеки или из другого пакета необходимо указать откуда его загружать. Для этого в начале файла до тела класса необходимо добавить `import java.util.Scanner`.

Пример:

```
public static void main(String[] args)
{
    // создаем объект класса сканер и передаем
    // в конструктор стандартный поток ввода
    Scanner scanner = new Scanner(System.in);
    System.out.println("Введите число");
```

```

// возвращает истину если введено целое число
boolean isInt = scanner.hasNextInt();
System.out.println(isInt);
// проверяем являлось ли введенное значение
// целым числом
if (isInt)
{
    // считываем из консоли значение целого числа
    // в переименую x
    int x = scanner.nextInt();
    System.out.println("Вы ввели число " + x);
}
else
{
    System.out.println("Это не целое число");
}
}

```

`System.in` — стандартный поток ввода с клавиатуры

2.12. Операции

Операции в языке java — это специальные символы, которые показывают, какое действие необходимо выполнить для заданных операндов.

Операнд — может быть литералом, переменной или выражением, над которыми производится операция.

Операции могут применяться к одному (унарная операция), двум (бинарная операция) или трем (тернарная операция) операндам.

Операции могут быть объединены в выражения.

В отличие от C++ в Java отсутствует перегрузка операций, задаваемая программистом. Создатели языка решили, что перегрузка операций существенно запутывает код и усложняет его понимание.

Есть несколько операций перегруженных по умолчанию:

- **+** — используется для сложения чисел и конкатенации строк;
- **&** — используется для побитовой операции с числами и для логического И;
- **|** — используется для побитовой операции с числами и для логического ИЛИ;
- **^** — используется для побитовой операции с числами и для логического исключающего ИЛИ;
- **==** — сравнивает любые типы;
- **!=** — сравнивает любые типы.

Арифметические операции

Обозначение	Описание	Пример
+	Унарный плюс, не меняет значение операнда	<pre>int i = -1; i = +i; System.out.println(i);</pre>
	Бинарный плюс, суммирует операнды	<pre>int i = -1; i = i + 3; System.out.println(i);</pre>
-	Унарный минус — меняет знак операнда на противоположный	<pre>int i = 1; i = -i; System.out.println(i);</pre>
	Бинарный минус — вычитает правый операнд из левого	<pre>int i = 3; i = i - 2; System.out.println(i);</pre>
*	Умножение операндов	<pre>int i = 2; i = i * 2; System.out.println(i);</pre>
/	Деление левого операнда на правый	<pre>int i = 4; i = i / 2; System.out.println(i);</pre>

Обозначение	Описание	Пример
%	Вычисление остатка от деления левого операнда на правый	<pre>int i = 3; i = i % 2; System.out.println(i);</pre>
++	Унарный оператор увеличения значения переменной на 1. Инкремент	<pre>i++; заменяет выражение i = i + 1;</pre>
--	Унарный оператор уменьшения значения переменной на 1. Декремент	<pre>i--; заменяет выражение i = i - 1;</pre>

Инкремент и декремент могут иметь одну из форм:

- префиксную (**++x**; // аргумент изменяется до его использования)
- постфиксную (**x++**; // аргумент изменяется после использования)

Примеры:

```
int k=4;
System.out.println(k++); // выводится 4 после вывода
                        // k=5

int j=4;
System.out.println(++j); // выводится 5 после вывода
                        // j=5
```

Побитовые операции

Побитовые операции производят вычисления, используя в качестве операндов побитовое представление числа (в двоичной системе счисления).

ВАЖНО!!! В качестве операндов в побитовых операциях могут быть использованы только целые примитивные типы или их классы-обертки.

Опера-ция	Описа-ние	Пример	Ре-зультат
&	И (AND)	В каждом разряде результата получаем значащий бит, в случае если присутствует значащий бит в левом и правом операнде. <pre>int i = 123 & 456;</pre> Битовое представление <pre>000000000000000000000000000000001111011 & 00000000000000000000000000000000111001000 = 000000000000000000000000000000001001000</pre>	72
	ИЛИ (OR)	В каждом разряде результата получаем значащий бит, в случае если присутствует значащий бит в левом или правом операнде. <pre>final int one = 123; final int two = 456; int i = one two;</pre> Битовое представление <pre>000000000000000000000000000000001111011 00000000000000000000000000000000111001000 = 0000000000000000000000000000000011111011</pre>	507
^	исключающее ИЛИ (XOR)	В каждом разряде результата получаем значащий бит, в случае если присутствует значащий бит в левом или правом операнде. Если бит есть в разряде обоих операндов в результате получаем 0. <pre>int i = 123 ^ 456;</pre> Битовое представление <pre>000000000000000000000000000000001111011 ^ 00000000000000000000000000000000111001000 = 00000000000000000000000000000000110110011</pre>	435
~	поразрядное отрицание (NOT)	Инвертирует все биты числа, на противоположные (включая знаковый бит) <pre>int i = ~123;</pre> Битовое представление: <pre>~ 000000000000000000000000000000001111011 = 111111111111111111111111111111110000100</pre>	-124
<<	сдвиг влево	Смещает биты левого операнда влево на количество бит указанном в правом операнде, заполняя справа нулем.	

Операция	Описание	Пример	Результат
>>	сдвиг вправо	Смещает биты левого операнда вправо на количество бит указанном в правом операнде, заполняя разряды справа нулем.	
>>>	без знаковый сдвиг вправо	Смещает биты левого операнда вправо на количество бит указанном в правом операнде, заполняя слева нулем игнорируя битовый сдвиг.	

`Integer.toBinaryString(value)` — преобразует целое число `value` в строку в побитовом представлении.

Пример метода выводящего число в побитовом представлении с дополнением не значимыми нулями:

```
private static void printToBinary(int value)
{
    System.out.println(String.format("%32s",
        Integer.toBinaryString(value))
        .replace(' ', '0'));
}
```

Операции сравнения

Результат операций сравнения всегда имеет тип `boolean`.

Обозначение	Условие	Пример
>	Операнд слева больше чем операнд справа	<code>int a = 7;</code> <code>boolean r = a > 5;</code>
<	Операнд слева меньше чем операнд справа	<code>int a = 7;</code> <code>boolean r = a < 5;</code>
>=	Операнд слева больше или равен операнду справа	<code>int a = 7;</code> <code>boolean r = a >= 5;</code>

Обозначение	Условие	Пример
<code><=</code>	Операнд слева меньше или равен операнду справа	<code>int a = 7;</code> <code>boolean r = a <= 5;</code>
<code>==</code>	Операнд слева в точности равен операнду справа	<code>int a = 7;</code> <code>boolean r = a == 5;</code>
<code>!=</code>	Операнд слева не равен операнду справа	<code>int a = 7;</code> <code>boolean r = a != 5;</code>

Логические операции

Логические операции применяются только к операндам типа `boolean`. Результат логических операций имеет тип `boolean`.

Обозначение операции	Описание
<code>&</code>	Логическое И
<code> </code>	Логическое ИЛИ
<code>^</code>	Логическое исключающее ИЛИ
<code> </code>	Оператор быстрой оценки ИЛИ*
<code>&&</code>	Оператор быстрой оценки И*
<code>!</code>	Отрицание

Примечание. Операторы быстрой оценки не вычисляют результат выражения в правом операнде, если результат операции можно определить по значению первого операнда.

Пример для И:

```
boolean t = 5 < 3 && 5 > 3
```

В данном примере в результате вычисления выражения левого операнда получим `false`, поэтому для вычисления

результата операции нет необходимости вычислять правый операнд.

Пример для ИЛИ:

```
boolean t = (2 == 2) || 3 != 2
```

В данном примере в результате вычисления выражения левого операнда получим **true**, поэтому для вычисления результата операции ИЛИ нет необходимости вычислять правый операнд.

В примере ниже использование укороченной логической операции позволяет избежать деления на 0.

Пример.

```
// Демонстрация использования укороченных логических
// операций
public class Runner {
    public static void main(String[] args){
        int number=50;
        int delitel=0;
        // второй операнд не вычисляется и ошибки
        // не возникает
        if(delitel!=0&&(number%delitel)==0)
            System.out.println(delitel+
                " is a divisor"+number);
        /* Без использования укороченного оператора
        возникнет ошибка "деление на ноль"
        java.lang.ArithmeticException: /by zero
        if(delitel!=0&(number%delitel)==0)
        System.out.println(delitel+" является делителем
        "+number); */
        System.out.println("End of the program");
    }
}
```

Таблица истинности логических операций

Операнд 1	Операнд 2	Операция			
			&	^	!Операнд1
false	false	false	false	false	true
true	false	true	false	true	false
false	true	true	false	true	true
true	true	true	true	false	false

Операция присваивания

Для сохранения в переменной значения литерала, переменной или значения выражения используется операция присваивания. Оператор присваивания имеет самый низкий приоритет, поэтому в начале выполняются все операции в выражении и только потом вычисленное значение передается в переменную.

Синтаксис:

```
переменная = выражение;
```

Пример:

```
int x = 3; // переменная будет инициализирована
           // значением 3

x = 6; // значение переменной будет перезаписано,
       // старое значение будет потеряно
```

Java позволяет присваивать значения одновременно нескольким переменным.

Пример:

```
int x;
int y;
int z;

x = y = z = 3;
System.out.println(x + y + z);
```

Результат: 9.

Тернарный оператор

Тернарный оператор содержит три операнда. Результатом операции может быть второй или третий операнд. Первый операнд должен иметь тип **boolean**.

Синтаксис:

```
выражение1 ? выражение2 : выражение3;
```

Операция вернет значение «выражение2» если результат вычисления «выражения1» примет значение истина, иначе вернет значение «выражения3».

Пример:

```
int x = 1;
int y = 2;
int a = (x != y) ? x : y; // для лучшего чтения кода
                        // рекомендуется брать
                        // условие в скобки.

System.out.println(a);
```

Результат: 1.

Приоритеты операций

Операции	Описание
() []	Круглые и квадратный скобки
++ (постфиксная) -- (постфиксная)	Постфиксный инкремент и декремент
++ (префиксная) -- (префиксная) + - ~ ! (тип)	Префиксный инкремент, декремент, унарный плюс, унарный минус, поразрядное отрицание, логическое отрицание, приведение типов
* / %	Умножение, деление, остаток от деления
+ -	Сложение, вычитание
>> << >>>	Побитовые сдвиги: вправо, влево, беззнаковый сдвг вправо
> >= < <= instanceof	Сравнение на: больше, больше или равно, меньше, меньше или равно, проверка принадлежности классу
== !=	Сравнение на равенство, сравнение на неравенство
&	Поразрядное и логическое И
^	Поразрядное и логическое исключающее ИЛИ
	Поразрядное и логическое ИЛИ,
&&	Логическое И быстрой оценки
	Логическое ИЛИ быстрой оценки
? :	Тернарная условная операция
= операция=	Операция присваивания, Комбинированные операции присваивания

Операции расположены в таблице в порядке убывания приоритетов сверху вниз. Операции в одной строке таблицы имеют одинаковый приоритет.

2.13. Управляющие конструкции

Для реализации алгоритма ветвления в коде в java присутствует два условных оператора, это оператор `if ... else` и оператор `switch`. Для реализации циклических алгоритмов есть три оператора, это `for`, `while`, `do..while`. Синтаксис и принцип работы похож на синтаксис и работу в языках C++ и C#.

Условные операторы

Синтаксис:

```
if (выражение) операция;
```

В круглых скобках может быть указано любое выражение, результатом которого должен быть тип `boolean`. В случае если в скобках выражение примет значение `true`, то будет выполнена «операция» следующая за круглыми скобками, иначе управление программы перейдет к следующей строке кода.

Пример:

```
int a = 3;  
if (a == 3) a = a + 2;  
System.out.println(a);
```

Результат: 5.

Если в результате выполнения условия в операторе `if` необходимо выполнить несколько операций, необходимо использовать «блочный разделитель» (фигурные скобки) для обозначения блока операций, которые будут выполнены.

Пример:

```
int a = 3;
if (a == 3) { // начало блока
    a = a + 2;
    a--;
// конец блока }
System.out.println(a);
```

Результат: 4.

Совместно с оператором **if** возможно использоваться ключевого слова **else**.

Синтаксис:

```
if (выражение) операция1;
else операция2;
```

В случае если в скобках «выражение» примет значение **false**, то будет выполнена «операция 2», «операция 1» выполнена не будет.

Пример:

```
int x = 3;
if (x > 3) System.out.println(++x);
else      System.out.println(--x);
```

Результат: 2.

Если нужно выполнить несколько команд по ветке **else**, они также объединяются фигурными скобками.

Пример:

```
int a = 3;
int b = 0;
```

```

if (a > 3) {
    b = a;
    System.out.println(a - 3);
}
else {
    b = -a;
    System.out.println(a);
}

```

Оператор **if** может использоваться совместно с **else** для создания логических цепочек.

Пример:

```

int age = 18;
if (age < 1) System.out.println("Грудничок");
else if (age >= 1 && age < 16) System.out.
    println("Ребенок");
else if (age >= 16 && age < 19)
    System.out.println("Подросток");
else if (age >= 19) System.out.println("Взрослый");

```

Для упрощения реализации алгоритма множественного выбора в java добавлен оператор **switch**. Оператор анализирует выражение в скобках и передает управление одному из сценариев (**case**). Далее программа выполнит весь код, находящийся в сценарии до конца оператора **switch**. Значения вариантов в **case** не должны повторяться.

Синтаксис:

```

switch (выражение)
{
    case вариант1:
        операция1;

```

```

    case вариант2:
        операция2;
    case вариант3:
        операция3;
    default:
        операция;
}

```

В скобках допустимо наличие значений примитивных типов `byte`, `short`, `char`, `int`¹.

Пример:

```

int x = 2;
switch(x)
{
    case 1:
        System.out.println(1);
    case 2: // будет выбран этот сценарий и выполнены
           // все команды до конца блока switch
        System.out.println(2);
    case 3:
        System.out.println(3);
}

```

Результат: 2 3.

Для прерывания сценария в блоке `switch` можно использовать ключевое слово `break`.

Пример:

```

int x = 2;
switch(x)

```

¹ Начиная с версии JSE 5.0 в качестве выражения можно использовать перечисления. Начиная с версии JSE 7.0 в качестве выражения можно использовать строки (String).

```

{
    case 1:
        System.out.println(1);
        break; // прерывание выполнение сценария,
              // управление передается в конец за блок switch
    case 2:
        System.out.println(2);
        break;
    case 3:
        System.out.println(3);
        break;
}

```

Результат: 1.

Ключевое слово **default** может использоваться в операторе **switch** в случае если ни один сценариев не был выполнен. В блоке **switch** может присутствовать только одно ключевое слово **default**. Порядок расположения блоков **case** и **default** не имеет значения, но принято располагать блоки **case** по возрастанию значения, а **default** в самом конце.

Пример:

```

int x = 5;
switch(x)
{
    case 3:
        System.out.println(3);
        break;
    case 1:
        System.out.println(1);
        break;
    case 2:
        System.out.println(2);
}

```

```

        break;
    default:
        System.out.println("default");
}

```

Результат: default.

Циклы

Для реализации циклического повторяющегося алгоритма в java есть операторы **while**, **do while** и **for**.

While – оператор цикла «пока»

Синтаксис:

```
while (выражение) операция;
```

Выражение в скобках (далее условие выполнения цикла) должно иметь тип **boolean**. В случае если условие выполнения цикла принимает значение истина (**true**), будет выполнена «операция» и управление передастся обратно в проверку выражения. Если значение ложно, выполнение тела цикла прекратится.

Пример:

```
int i = 0;
while (i < 5) System.out.println(i++);
```

Результат: 0 1 2 3 4.

Телом цикла называется оператор, который будет повторяться в цикле. Он располагается в конструкции **while** сразу после условия выполнения цикла, заключенного в круглые скобки. Если в цикле нужно повторять

несколько операторов, то они объединяются с помощью фигурных скобок в один составной оператор.

Пример:

```
int i = 0;
while (i < 5) { // начало тела цикла
    i++;
    System.out.println(i);
} // конец тела цикла
```

Результат: 1 2 3 4 5.

Примечание. Если условие выполнения цикла изначально ложно, тело цикла ни разу не будет выполнено.

Пример:

```
int i = 0;
while (i > 5) {
    System.out.println(i++);
}
System.out.println(i);
```

Результат: 0.

Если члены условия цикла не меняются внутри тела цикла, а условие выполнения цикла истинно, такой цикл будет выполняться бесконечно.

Пример бесконечного цикла:

```
while (true) {
    System.out.println("infinity");
}
```

do-while — цикл с постусловием

Синтаксис

```
do операция;  
while (выражение);  
do {  
    операция1;  
    операция2;  
} while (выражение);
```

Отличие **do while** от цикла **while** состоит в том, что тело цикла **do while** гарантированно выполнится один раз, до проверки условия выполнения цикла. Повторное выполнение тела цикла будет зависеть от значения выражения, если оно истинно, то программа передаст управление в начало тела цикла.

Пример:

```
int i = 0;  
do {  
    i++;  
    System.out.println(i);  
} while (i > 10);
```

Результат: 1.

For — выполнить цикл «n — раз»

Синтаксис:

```
for (<инициализация>; <условие>; <счетчик>) операция;
```

Блок «инициализация» может содержать объявление и инициализацию переменных, которые будут доступны в теле цикла. Заполнение блока не обязательно.

Блок «условие» должен содержать выражение результатом которого должно быть значение типа `boolean`. Заполнение блока не обязательно.

Блок «счетчик» может содержать любые операции. Заполнение блока не обязательно.

Пример все блоки заполнены

```
for (int i = 0; i < 10; i++)
{
    System.out.println(i);
}
```

Пример с отсутствием объявления переменной счетчика (необходим если нужен доступ к счетчику за пределом тела цикла):

```
int i;
for (i = 0; i < 10; i++)
{
    System.out.println(i);
}

i+=2;
System.out.println(i);
```

Пример с отсутствием блока инициализации

```
int i = 0;

for (; i < 10; i++)
{
    System.out.println(i);
}
```

Пример с использованием нескольких счетчиков:

```
for (int i = 1, j = 3; i < 5 & j > 0; i++, --j)
{
    System.out.println("i = " + i + " j = " + j);
}
```

Результат: $i = 1 \quad j = 3$
 $i = 2 \quad j = 2$
 $i = 3 \quad j = 1$.

For-each

В java нет специального ключевого слова для перебора элементов массива или коллекций, как в других языках. Для реализации цикла for-each используется специальный синтаксис оператора for.

Синтаксис:

```
for (тип элемент : коллекция) {
    операция;
}
```

Пример для массива:

```
int [] numbers = new int[] { 3, 2, 1 };
for (int number : numbers)
{
    System.out.print(number + " ");
}
```

Пример для коллекции:

```
List<Integer> lists = new ArrayList<Integer>();
lists.add(1);
```

```
lists.add(3);
for (Integer value: lists)
{
    System.out.println(value);
}
```

Ограничением этой формы цикла является тот факт, что элементы массива или коллекции перебираются строго последовательно. Например, нельзя пройти только по элементам с нечетными индексами, либо от конца массива к началу.

Кроме того, в цикле `for each` элементы массива доступны *«только для чтения»*, т.е. их нельзя изменить (поскольку итерационная переменная является копией элемента).

Операторы *break* и *continue*

Оператор `break` прерывает выполнение цикла и передает управление в конец тела цикла.

Пример:

```
int i = 0;
while (i < 10)
{
    System.out.println(i);
    if (i >= 2) break;
    i++;
}
```

Результат: 0 1.

Оператор `break` может быть использован совместно с меткой, например для одновременного прерывания

нескольких вложенных циклов. В этом случае управление будет передано в конец блока помеченного меткой указанной после **break**. Метка именует блок кода.

Пример:

```
two: for (int n = 0; n < 10; n++) {
    for (int j = 10; j > 0; j--) {
        System.out.print(j - n + " ");
        if (j + n == 5 && n > 0)
            break two;
    }
}
```

Оператор **continue** прерывает выполнения тела цикла и передает управление в начало цикла. Происходит пропуск текущей итерации цикла.

Пример:

```
int i = 0;
while (i < 10)
{
    i++;
    if (i % 2 == 0)
        continue;
    System.out.println(i);
}
```

Результат: 1 3 5 7 9.

Оператор **continue** так же может применяться с меткой.

Пример:

```
one: for (i = 0; i < 10; i++) {
    for (int j = 0; j < 10; j++) {
```

```

        if (i == j) continue one;
        System.out.printf("i=%d j=%d \n", i, j);
    }
}

```

2.9. Методы

Метод аналогичен функции языка C++ и представляет собой законченный фрагмент кода, к которому можно обратиться по имени. Он описывается один раз, а вызываться может столько раз, сколько необходимо. Один и тот же метод может обрабатывать различные данные, переданные ему в качестве аргументов.

Синтаксис метода:

```

[спецификаторы]тип имя_метода ([список_параметров])
{
    // тело_метода
}

```

Здесь тип обозначает конкретный тип данных, возвращаемых методом. Если метод не возвращает значение, то следует указать тип **void**.

Имя метода — это любой допустимый идентификатор, кроме тех, которые уже используются другими элементами кода в текущей области действия.

Список параметров представляет собой последовательность пар «тип идентификатор», разделенных запятыми. Параметр — это значение, которое передается внутрь метода. Список параметров может быть пустым.

Пример

Объявим метод, который находит максимальное из трех вещественных чисел.

```
static double maximum(double x, double y, double z){
    double max=x>y?x:y;
    max=z>max?z:max;
    return max;
}
```

Выход из метода и возврат значения в вызывающую программу выполняет оператор

```
return значение;
```

Если тип метода **void**, то в теле этого метода используется **return** без параметров, либо он может вообще отсутствовать. В последнем случае возврат в вызвавшую программу происходит при достижении последней закрывающей скобки метода.

Если внутри метода объявляются переменные, то они являются локальными. Они не инициализируются значениями по умолчанию. При завершении работы метода эти переменные становятся недоступны.

Параметры метода — это также локальные переменные, которые создаются при входе в метод и получают копии значений соответствующих аргументов в его вызове. Список аргументов в вызове как бы накладывается на список параметров, поэтому они должны попарно соответствовать друг другу.

Аргументы в метод передаются по значению. Всегда! (В отличие от других языков, в Java нет передачи параметра

по ссылке!) По окончании работы метода его параметры уничтожаются, а при следующем вызове — создаются вновь.

Пример

Организуем вызов описанного ранее метода `maximum()` из главного метода `main()`.

```
public class Runner{
    public static void main(String[] args) {
        double a=0.5,b=17.2,c=-34;
        double result=maximum(a,b,c);
        // локальная переменная другого метода
        // недоступна
        // System.out.println(max); //ошибка
        // параметр другого метода недоступен
        // System.out.println(x); //ошибка
        System.out.println("Maximum= "+result);
    }
}
```

При вызове метода переменные `x`, `y` и `z` (локальные параметры) получают копии значений переменных `a`, `b` и `c` (аргументов). После возврата из метода его параметры и локальные переменные недоступны.

ВАЖНО! *Принцип передачи по значению относится именно к самому параметру метода. В том случае, когда параметр является ссылкой, метод получает копию ссылки, но при этом может изменить сам объект!*

Пример.

Метод `addExention()` получает в качестве параметра копию ссылки на объект `StringBuilder` (изменяемую стро-

ку). Пользуясь этой ссылкой, он изменяет содержимое объекта (добавляет расширение «.txt»).

```
public class Runner {
    public static void main(String[] args) {
        // создание объекта
        StringBuilder filename=new StringBuilder("data");
        addExtention(filename); // передача в метод
                                // ссылки на объект
        System.out.println("Filename with extension:" +
                           filename);
    }
    static void addExtention(StringBuilder sb){
        sb.append(".txt"); // добавление символов в строку
    }
}
```

На рисунке показано состояние памяти после вызова метода `addExtention()`:

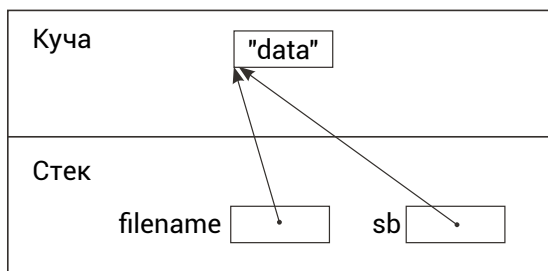


Рисунок 29.

Параметр `sb` получил копию ссылки `filename`. Изменения, сделанные с использованием этой ссылки, будут доступны и в `main()`. Сам же параметр `sb` при выходе из метода будет уничтожен. Имя метода вместе с количеством, типами и спецификаторами его параметров представляет

собой **сигнатуру метода**. В классе не должно быть методов с одинаковыми сигнатурами.

В языке Java, аналогично языку C++, может быть реализована перегрузка методов. Т.е. могут быть объявлены несколько методов с одним именем, которые различаются набором параметров (могут отличаться количество параметров, их типы, последовательность, спецификаторы). При вызове метода на исполнение анализируется список фактических параметров и выбирается метод, сигнатура которого соответствует этому списку. Именно он и исполняется.

Перегрузка является одним (наиболее простым) из проявлений полиморфизма, реализуя принцип «Один интерфейс — несколько методов».

Методы в языке Java делятся на статические (со спецификатором `static`) и методы экземпляра. Статические методы вызываются от имени класса, а методы экземпляра — от имени объекта класса. Если вызов метода выполняется внутри самого класса, к методу можно обращаться просто по имени.

Статический метод, по сути, аналогичен функции языка C. Из статического контекста без создания экземпляра какого-либо объекта можно вызвать только статический метод.

Пример.

```
public class Example
{
    public static void main(String[] args)
    {
```

```

double a=0.6;
double b=1.25;
// вызов статического метода — вариант 1
double result1=Example.product(a,b);
// вариант 2-вызов стат. метода внутри
// того же класса
double result2=product(a,b);
// создание экземпляра класса
Example ex=new Example();
// вызов метода экземпляра от имени объекта
double result3=ex.summa(a,b);
// статический метод
static double product(double x, double y)
{
    return x*y;
}
// метод экземпляра
double summa(double x, double y)
{
    return x+y;
}
}

```

Другие спецификаторы методов (**public**, **private**, **final** и др.) будут рассмотрены в модуле 2.

В Java, как и в других языках программирования, возможно использование рекурсивных алгоритмов. **Рекурсией** называется прием программирования, когда метод вызывает сам себя (точнее не себя, а другую копию этого же метода). Классический пример рекурсии — вычисление факториала $n!=1 \times 2 \times \dots \times n$.

Пример.

```

public class Runner {
    public static void main(String[] args) {
        int k=5;
    }
}

```

```

    double result=fact(k);
    System.out.println("Factorial "+k+
                       " is equal "+result);
}

static double fact(int n){
    if(n<=1) return 1; //начало возвратов
    return fact(n-1)*n; //рекурсивный вызов
}
}

```

Математические функции в Java

Математические функции в Java реализованы в виде статических методов класса **Math**. Это означает, что они должны вызываться от имени класса, например: **Math.sin(x)** или **Math.pow(x,3)**.

Класс **Math** входит в пакет `java.lang`, который подключается по умолчанию. Поэтому его не нужно специально импортировать в начале программы (как, например, класс **Scanner**).

Наиболее употребительные методы класса **Math**:

- **sqrt(x)** — извлекает квадратный корень из числа x .
- **pow(x, y)** — x возводится в степень y (оба параметра могут быть вещественными).
- **sin(x), cos(x), tan(x)** — тригонометрические функции $\sin$, $\cos$ и $\tan$ угла x , указанного в радианах.
- **asin(x), acos(x), atan(x)** — обратные тригонометрические функции, возвращают угол в радианах.
- **exp(x)** — возвращает значение экспоненты, возведенной в степень x .

- `log(x)` — возвращает значение натурального логарифма числа `x`.
- `log10(x)` — возвращает значение десятичного логарифма числа `x`.
- `abs(x)` — возвращает модуль числа `x`.
- `round(x)` — округляет вещественное число до ближайшего целого.
- `random()` — возвращает вещественное случайное число от 0 до 1.

Многие методы этого класса перегружены, так что тип результата зависит от типа аргумента. Например, если аргумент метода `abs` имеет тип `double`, то и результат — также тип `double`. А если тип аргумента `int`, то и результат его будет `int`.

Кроме методов, в классе `Math` имеется две константы: `Math.E` (2,71828 — основание натурального логарифма) и `Math.PI` (3,141592... — число Пи).

Пример использования методов класса `Math` для вычисления функции $y = \frac{\sqrt{2}}{2} \sin\left(\frac{x}{2} + \pi\right)$ приведен ниже.

```
import java.util.Scanner;
public class Runner {
    public static void main(String[] args) {
        double x,y;
        Scanner scan=new Scanner(System.in);
        System.out.print("Enter an argument: ");
        x=scan.nextDouble();
        y=Math.sqrt(2)/2*Math.sin(x/2 +Math.PI);
        System.out.println("Function value= "+y);
    }
}
```

2.10. Основные положения Java Code Convention

[Code Conventions for the Java Programming Language](#) — это соглашение по оформлению кода. Соблюдение положений конвенции делает Ваш код более читаемым и понятным для других программистов. Эти положения являются рекомендациями, которые составлены на основе личного опыта при работе с кодом программистов.

Имена классов

Фалы с исходным кодом должны иметь расширение java. Файлы с байт-кодом должны иметь расширение class.

Организация файла исходного кода

Размер файла не должен превышать 2000 строк. Файлы с большим количеством строк затрудняют поиск нужного блока кода, увеличивают время компиляции и время загрузки класса в java машину.

Файл должен разделяться на секции, которые отделяются друг от друга пустой строкой.

В одном файле может находиться только один класс с модификатором **public** (основной класс).

Не желательно размещение в одном файле других классов (не public).

Порядок следования секций в классе:

- JavaDoc;
- Заголовок класса;
- Статические поля;
- Не статические поля;
- Конструкторы;
- Методы.

Отступы

Единица отступа 4 пробела (может использоваться табуляция). Длина строки — не более 80 символов.

Именованние классов и интерфейсов

Классы должны именоваться с большой буквы, английскими символами без пробелов. Имя класса должно соответствовать одному объекту этого класса.

Пример:

```
class Cats // плохо
class Cat  // хорошо
```

Если в имени класса содержится более одного слова, каждое слово выделяется большой буквой (Camel нотация). Имя класса должно соответствовать

Пример:

```
class TileIterator
```

Интерфейсы должны именоваться по правилам классов. Не желательно использование дополнительных указаний в имени интерфейса на принадлежность к интерфейсу.

```
interface ICloneable // плохо
interface Cloneable  // хорошо
```

Если в имени класса содержится аббревиатура содержащая больше двух букв, все буквы аббревиатуры кроме первой должны быть строчными.

Пример:

```
class FPSRenderer // плохо
class FpsRenderer // хорошо
```

Именованние переменных

Объявление (декларация) нескольких переменных должно быть в разных строках.

```
int a, b; // плохо
int a; // хорошо
int b; // хорошо
```

Если возможно инициализировать переменные в месте объявления.

```
int a;
...
a = 255; // плохо
int a = 255; // хорошо
```

Необходимо объявлять переменные как можно ближе к месту использования

```
int a = 255; // плохо

System.out.println("file")
a + = 127;

System.out.println("file")
int a = 255; // хорошо
int c = a + 127;
```

Массивы

При объявлении массива необходимо указывать скобки после указания типа переменной, а не после имени.

```
int a [] = new int[3]; // плохо
int[] a = new int[3]; // хорошо
int [] a [] = new int[3]; // плохо
int[][] a = new int[3][3]; // хорошо
```

Константы

Константы принято именовать большими буквами, а разделителем использовать символ подчеркивания.

Примеры:

```
final int MAX_STEP = 3; // хорошо
final String DELIMITER = ";"; // хорошо
```

Методы

Методы должны именоваться латинскими буквами начиная с маленькой буквы. Первое слово в названии метода должно быть глаголом или наречием. Если в имени метода содержится более одного слова, каждое слово кроме первого выделяется большой буквой (Camel нотация).

Примеры:

```
void startprocess() {} // плохо
void Start() {} // плохо
void start_process() {} // плохо
void sProcess() {} // плохо
void sp() {} // плохо
void start() {} // хорошо
void startProcess() {} // хорошо
```

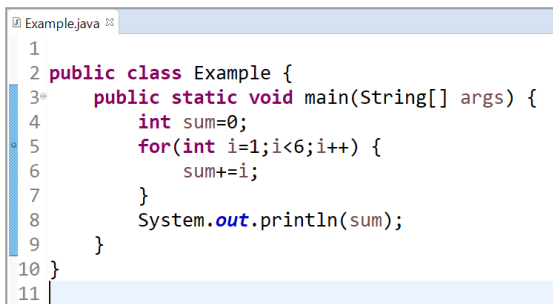
2.11. Отладка программ в Eclipse

Кроме ошибок, выявляемых на этапе компиляции, код часто содержит логические ошибки, которые проявляют себя только на этапе выполнения программы. При наличии таких ошибок программа может зависать, выбрасывать незапланированные исключения или просто будет работать не так, как это задумывалось.

Обнаружить такие ошибки помогает встроенный отладчик «Eclipse». Он позволяет:

- остановить выполнение программы в определенном месте кода,
- посмотреть состояние переменных,
- выполнять программу пошагово, останавливаясь после каждого оператора,
- и многое другое.

Рассмотрим использование отладчика «Eclipse» на примере, в котором находится сумма целых чисел от 1 до 5. Перед запуском отладчика устанавливаем точку прерывания, выполнив **двойной щелчок** по голубой полосе слева от кода, напротив цикла **for**. Появится небольшая точка (рис. 30):



```
1
2 public class Example {
3     public static void main(String[] args) {
4         int sum=0;
5         for(int i=1;i<6;i++) {
6             sum+=i;
7         }
8         System.out.println(sum);
9     }
10 }
11
```

Рисунок 30.

Аналогично щелчком по точке прерывания можно ее убрать.

Если запускать программу на выполнение обычным способом (например, «Ctrl+F11»), то эта точка прерывания не окажет никакого влияния. Встроенный отладчик запускается, если задать команду «Run/Debug» (или нажать «F11»).

При этом «Eclipse» предлагает переключиться на другую перспективу, оптимизированную для отладки программ (рис. 31).

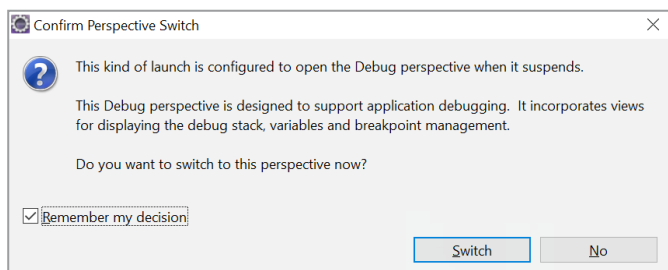


Рисунок 31.

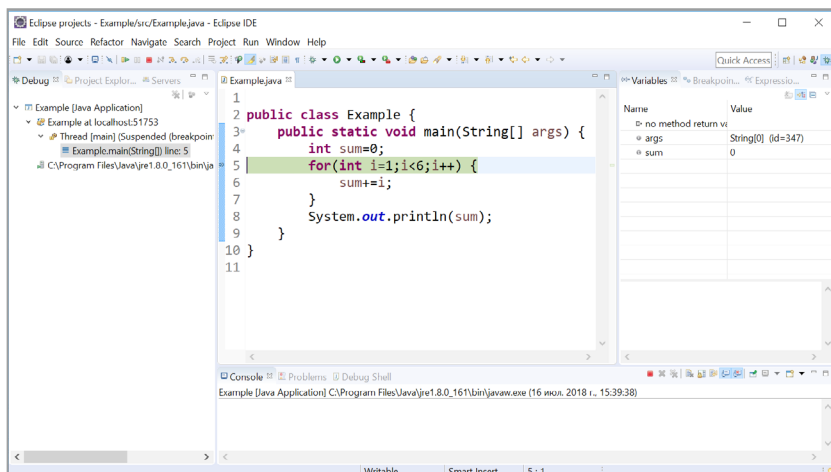


Рисунок 32.

Нажав кнопку «Switch», согласимся с этим.

Программа будет выполнена до точки прерывания (выполнение остановится перед оператором, отмеченным этой точкой). При этом перспектива изменится, как показано на рисунке (рис. 32).

Внизу экрана будут знакомые нам окно консоли (*Console*) и окно вывода ошибок компиляции (*Problems*).

Слева находится панель «Debug» (*Отладка*), в которой отражено, в каком месте программы произошла остановка. Указывается имя приложения, стек вызовов методов и номер строки в методе. Также указывается поток выполнения (или нить — *Thread*).

Справа открыта панель «Variables» (*Переменные*). В ней показаны все локальные переменные того метода, который выполняется в текущий момент. Указаны их имена и значения. Если выделить какую-либо переменную в этой панели, то соответствующее значение появляе-
ется в нижней части окна. А если курсор поставить на значение переменной в столбце *Value*, то это значение можно изменить.

Вместо панели «Variables» можно переключиться на панель «Breakpoints» (*Точки прерывания*) (рис. 33).

Здесь можно просмо-
треть все точки прерыва-

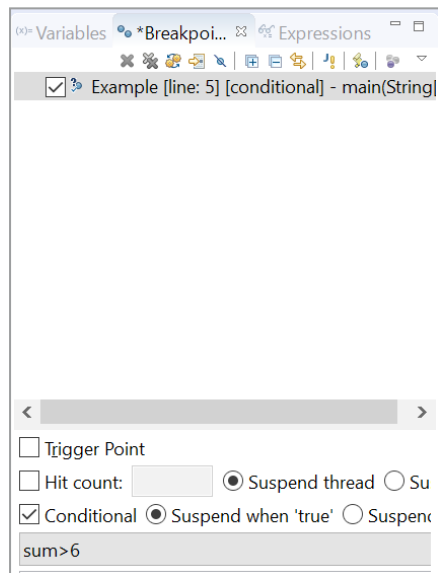


Рисунок 33

ния и управлять ими. Например, убрав флажок напротив какой-либо точки, мы ее деактивируем (отключаем). Поставив флажок, делаем эту точку прерывания вновь активной.

В этой панели можно также задать условие для точки прерывания (*Conditional breakpoint*). Например, на рисунке выше задано условие, что прерывание программы произойдет только в случае, если переменная **sum** станет больше 6. Нужно сохранить созданное условие, нажав «Ctrl+s».

И еще одна вкладка в правой панели — окно «Expressions» (Выражения):

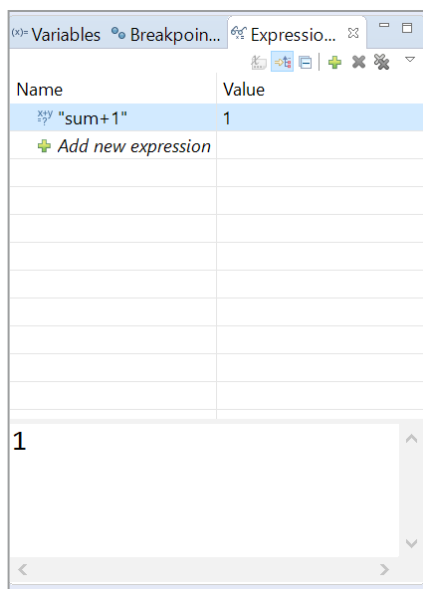


Рисунок 34.

Здесь можно вводить различные выражения с участием переменных и получать их значения.

Для выполнения каких-либо действий в процессе отладки можно воспользоваться меню «Run» (либо горячими клавишами, либо соответствующими кнопками на панели инструментов):

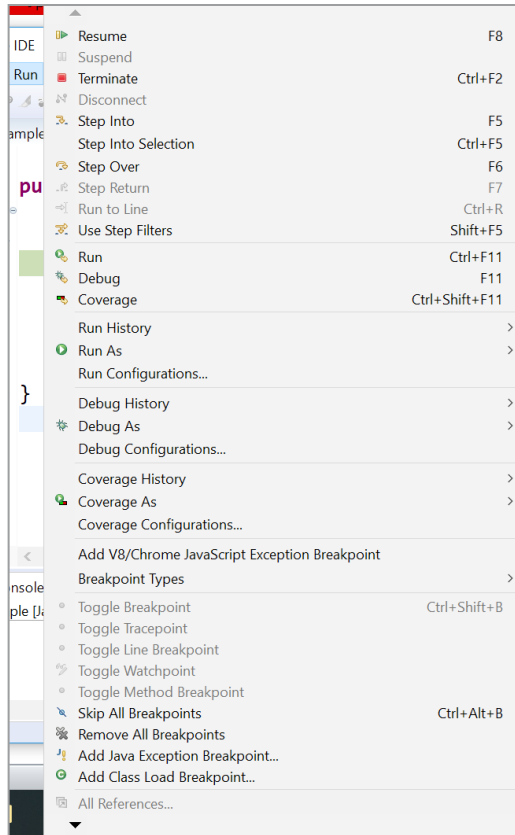


Рисунок 35.

Основные горячие клавиши отладчика:

- **F5 (Step into)** — выполнение одного оператора кода. Если при этом должен быть выполнен вызов метода, то пошаговое выполнение заходит внутрь него.

- **F6** (*Step Over*) — также выполнение одного оператора. Но вызов метода при этом выполняется как один шаг, без захода внутрь.
- **F8** (*Resume*) — продолжение выполнения до следующей точки прерывания либо до конца программы.
- **Ctrl+F2** (*Terminate*) — срочное завершение выполнения программы.

Выполняя пошагово программу, можно наблюдать за изменением значений переменных в окне «Variables». Также можно подвести курсор к имени переменной в коде и увидеть ее значение во всплывающем окне. И еще один способ просмотра значения переменной — из контекстного меню к имени переменной выбрать команду «Inspect».

Команда «Watch» из того же контекстного меню позволяет быстро добавить переменную в окно «Expressions».

3. Массивы и строки

3.14. Массивы

Массив в Java — это конечный набор элементов одного типа. Массив в Java это объект, поэтому переменная, указывающая на массив, является переменной ссылочного типа.

Одномерные массивы

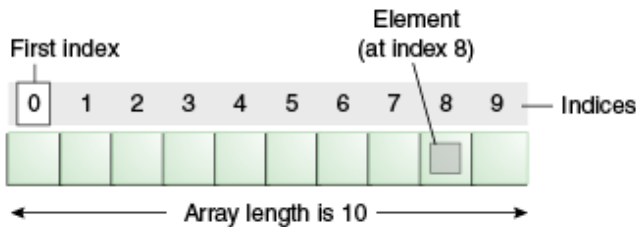


Рисунок 36.

Синтаксис объявления ссылки массива.

```
тип[] идентификатор;
```

ИЛИ

```
тип идентификатор[];
```

Пример объявления ссылки на массив:

```
int[] mas; // объявляется переменная типа массив
           // с именем mas
```

ИЛИ

```
int mas[];
```

Объект массива создается с помощью оператора **new**. Таким образом, все массивы в Java создаются **динамически**.

Синтаксис создания массива элементов:

```
new int[<количество элементов>;
```

Пример:

```
int[] mas; // объявление переменной массива
mas = new int[3]; // создание массива
```

ИЛИ

```
int[] mas = new int[3]; // создан массив на три
// элемента и ссылка на объект записана в
// переменную с именем mas.
```

Состояние памяти после выполнения компилятором этого кода показано на рисунке:

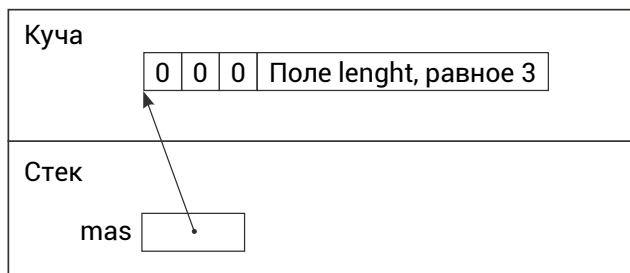


Рисунок 37.

ВАЖНО!!! Сразу после создания массива его элементы инициализируются значениями по умолчанию, для всех числовых типов это 0, для *char* символ с юникодом 0, для *boolean* — *false*, для ссылочных типов *null*.

Обращение к элементам массива происходит по индексу. Индексация элементов массива начинается с 0.

Пример:

```
int[] mas = new int[3];  
System.out.println(mas[0]);
```

Результат: 0.

Значение элементов массива можно установить в момент создания массива. Этот процесс называется инициализация массива.

Пример:

```
int[] mas = new int[]{3, 2, 1}; // будет создан массив  
// на три элемента, его первый элемент будет  
// иметь значение 3, второй 2, третий 1.
```

ВНИМАНИЕ!!! Нельзя одновременно инициализировать массив и указывать его размер при создании, так как его размер задается количеством элементов в блоке инициализации.

Пример:

```
int[] mas = new int[5]{3, 2, 1}; // ошибка компиляции!!!
```

Для массивов существует упрощенная запись инициализации массива.

Пример:

```
int[] mas = {3, 2, 1};
```

Массив имеет поле `length` в котором храниться длина массива (количество элементов заданных при создании массива), изменить это свойство нельзя.

Пример:

```
int[] mas = {3, 2, 1};
mas.length = 5; // ошибка компиляции
System.out.println(mas.length);
mas = {1, 2, 3, 4, 5}; // ошибка компиляции, можно
                        // использовать только при
                        // объявлении.
```

Обычно поле `length` используется для перебора элементов массива:

```
Random rand=new Random();
for(int i=0;i<a.length;i++){
    a[i]=rand.nextInt(10)+1;
    System.out.print(a[i]+" ");
}
```

В этом примере также показано, как можно инициализировать массив случайными числами от 1 до 10. Создается объект `rand` класса `Random` (начальное значение генератора псевдослучайных чисел будет всегда уникальное). Затем в цикле производится обращение к методу `nextInt(10)`, который выдает очередное псевдослучайное целое число от 0 до 9.

Чтобы получить целое случайное число в диапазоне от `a` до `b`, следует задать `a[i]=rand.nextInt(b-a+1)+a;`

Для сортировки массивов можно использовать статический метод `sort()` класса `Arrays`: `Arrays.sort(a);`

Метод `sort()` может применяться только к таким массивам, для элементов которого реализована операция сравнения (на больше-меньше).

Пример.

Сформируйте массив из 15 элементов, заполнив его целыми случайными числами от -5 до 15. Определите индекс первого максимального элемента. Сформируйте новый массив, переписав в него элементы исходного массива, расположенные после найденного максимума.

Выполните сортировку исходного массива по возрастанию элементов.

Решение задачи приведено ниже. Также показан результат работы программы (один из возможных).

```
import java.util.Arrays;
import java.util.Random;
public class Runner {
    public static void main(String[] args) {
        int[] a=new int[15]; //создание массива
        // инициализация генератора псевдослучайных
        // чисел
        Random rand=new Random();
        // инициализация случайными числами и вывод
        // на консоль
        System.out.println("Source array");
        for(int i=0;i<a.length;i++){
            a[i]=rand.nextInt(21)-5;
            System.out.print(a[i]+" ");
        }
        System.out.println();
        // поиск индекса максимума
        int imax=0;
        for(int i=0;i<a.length;i++){
```

```

        if(a[i]>a[imax]) imax=i;
    }

    // определение количества элементов нового
    // массива
    int k=a.length-imax-1;
    // формирование нового массива
    if(k==0) {
        System.out.println("The new array
                           is not formed");
    }
    else{
        int[] b=new int[k]; // выделение памяти
        int j=0; // индекс в новом массиве
        for(int i=imax+1;i<a.length;i++){
            b[j++]=a[i];
        }

        // печать сформированного массива
        // цикл for each
        System.out.println("An array of elements
                           after the maximum is
                           formed ");

        for(int x:b){
            System.out.print(x+" ");
        }
        System.out.println();
    }

    Arrays.sort(a); // сортировка
    // вывод отсортированного массива —
    // цикл for each
    for(int x:a){
        System.out.print(x+" ");
    }
    System.out.println();
}
}

```

Результат работы программы:

Source array: 5 9 4 8 9 8 3 8 8 15 -5 13 9 15

An array of elements after the maximum is formed:

-5 13 9 15

-5 3 4 5 8 8 8 8 9 9 13 15 15

Многомерные массивы

Многомерных массивов в языке java нет. Для имитации работы с данными в n мерном пространстве в java используется вложение массивов.

Для создания многомерных массивов при объявлении добавляются дополнительные квадратные скобки.

Пример создания двухмерного массива:

```
int[][] mas = new int[5][5]; // создается массив
// на 5 элементов, где элементами являются другие
// массивы на 5 элементов.
```

Пример создания трехмерного массива:

```
int[][][] mas = new int[5][3][3];
```

Структура двухмерного массива в java (рис. 38).

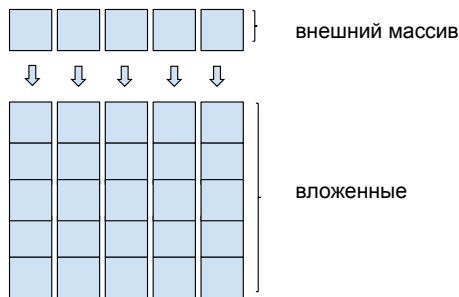


Рисунок 38.

Обратится к элементу многомерного массива можно используя несколько индексов.

Пример записи значения в квадратный¹ массив:

```
int[][] mas = new int[3][3]; // создается массив на 5
mas[2][1] = 3; // запись значения в массив
```

Синтаксис многомерных массивов предусматривает отложенную инициализацию.

Пример:

```
int[][] mas = new int[3][]; // создается одномерный
                          // массив из трех элементов значения
                          // которых null

mas[0] = new int[4];
mas[1] = new int[]{3, 3};
mas[2][2] = 7; // ошибка во время выполнения
```

Использование цикла *for-each* для работы с массивами

Пример вывода значений массива:

```
int [] numbers = new int[] { 3, 2, 1 };

for (int number : numbers)
{
    System.out.print(number);
}
```

¹ Квадратным называют двухмерный массив у которого количество элементов в вложенных массивах равно количеству элементов в исходном массиве.

Пример:

```
int [] numbers = new int[] { 3, 2, 1 };

for (int number : numbers)
{
    number = 3; // переменная number может быть
                // использована только для чтения
                // значений элементов массива
}

System.out.print(Arrays.toString(numbers));
```

Результат: [3, 2, 1]

Метод `java.lang.System.arraycopy()`

Метод `arraycopy` копирует значения из одного массива в другой. Метод нативный¹ (написан на языке C++) и благодаря этому копирует значения из одного массива в другой быстрее любого кода написанного с использованием языка `java`. Используется в коллекциях например в `java.util.ArrayList`.

Синтаксис:

```
System.arraycopy(src, srcPos, dest, destPos, length),
```

где

- `src` массив откуда копируются значения,
- `dest` массив куда копируются значения,
- `length` количество копируемых элементов из массива `src`,
- `srcPos` индекс в массиве `src` с которого начинается копирование,
- `destPos` индекс в массиве `dest` в который начнут копироваться элементы.

¹ Нативный метод — имеет реализацию на другом языке, не на `java`.

Пример:

```
int[] src = new int[]{1, 2, 3, 4, 5};
int[] dest = new int[]{5, 4, 3, 2, 1};
int length = 3;
int srcIndex = 1;
int destIndex= 2;

System.arraycopy(src, srcIndex, dest,
                 destIndex, length);
System.out.println(Arrays.toString(dest));
```

Результат: [5, 4, 2, 3, 4].

Примечание. Если указанное количество элементов отсутствует в исходном массиве или в массиве приемнике возникнет исключительная ситуация.

Класс `java.util.Arrays`

Класс `Arrays` содержит статические методы для работы с массивам.

- **`toString`** — метод преобразует содержание массива в строку. Если в качестве элементов массив содержит объекты, то для каждого элемента будет вызван метод `toString()` и полученные строки будут конкатенированы¹.

Пример вывода содержимого массива в консоль:

```
int [] mas = new int[]{1,2,3};
String content = Arrays.toString(mas);
System.out.println(content );
```

¹ Конкатенация — объединение нескольких строк в одну строку.

Массив с точки зрения java является объектом специального класса. Как и все классы в java он наследуется от класса `Object`, но метод `toString()` в нем не переопределен, поэтому при передачи его в метод `System.out.println` мы видим стандартную реализацию метода `toString()`. Поэтому для вывода на консоль многомерных массивов необходимо использовать метод `Arrays.deepToString()`.

Пример вывода на консоль содержимого многомерного массива:

```
String [][] objects = new String[3][3];
String content = Arrays.deepToString(objects);
System.out.println(content );
```

- **fill** — метод заполняет массив одинаковыми значениями.

Пример:

```
boolean[] test1 = new boolean[3];
Arrays.fill(test1, true);
```

Пример заполнения элементов массива в диапазоне с 2 по 5 значением 2:

```
int[] mas = new int[10];
int startIndex = 1;
int endIndex = 4;
Arrays.fill(mas, startIndex, endIndex, 2);
```

- **sort** — сортирует значения массива в естественном порядке (от меньшего к большему).

Пример:

```
int [] mas = new int[] {3, 1, 4, 6, 2};
Arrays.sort(mas);
System.out.println(Arrays.toString(mas));
```

Результат: [1, 2, 3, 4, 6]

Если в массиве хранятся элементы ссылочных типов, в методе `sort` есть возможность задать правила сортировки с использованием объекта класса реализующего интерфейс `Comparator`.

Пример сортировки по убыванию:

```
Integer [] mas = new Integer[] {3, 1, 4, 6, 2};
Arrays.sort(mas, Collections.reverseOrder());
System.out.println(Arrays.toString(mas));
```

Результат: [6, 4, 3, 2, 1]

- **equals** — сравнивает содержимое двух массивов (поэлементно) и если количество элементов равно и все элементы эквиваленты, возвращает истину, иначе ложь.

Пример:

```
int [] mas1 = {1,2,3};
int [] mas2 = {1,2,3};
boolean isEqual = Arrays.equals(mas1, mas2);
System.out.println(isEqual);
```

- **binarySearch** — возвращает индекс под которым был найден элемент, используя бинарный поиск.

Пример:

```
int index = Arrays.binarySearch(mas, 3);
```

- **deepHashCode** — возвращает хэш-код вычисленный на основе глубокого анализа элементов массива. Можно использовать для массивов хранящих ссылочные типы.
- **asList** — возвращает не изменяемый список (объект адаптер) преобразующий элементы массива в элементы списка.

3.15. Строки

Класс **String** — это самый часто используемый класс, предназначенный для хранения набора (массива) символов. Состояние объектов класса **String** невозможно изменить после создания объекта (объекты класса являются неизменяемыми). Следуя из объявления класса: **public final class String** запрещено наследовать подклассы от класса **String**.

Класс **String** реализует три интерфейса **Serializable** (сериализация), **Comparable<String>** (сравнение), **CharSequence** (последовательность символов). Класс **String** в сущности является оберткой над массивом символов (**char [] value**).

Внутреннее устройство класса **String**.

```
public final class String{
    private final char value[]; // массив символов
    private final int offset;   // смещение от начала
                                // массива
    private final int count;    // количество символов
                                // в строке
    private int hash;           // хэш код строки
}
```

Создать объект строки в java можно с помощью оператора **new** или строкового литерала (символы ограниченные двойными кавычками).

Пример с строковым литералом:

```
String name = "Вася";  
name = ""; // пустая строка
```

Пример с оператором new:

```
String name = new String("Вася");  
name = new String(); // пустая строка
```

При первом использовании строкового литерала в программе в памяти создается новый объект класса **String** и ссылка на него присваивается переменной. Объект строки создается в специально отведенном месте памяти, называемом пул строк¹. При повторном использовании литерала, новый объект не создается, а в переменную присваивается ссылка на ранее созданный объект.

Пример:

```
String name1 = "Вася"; // создается объект новой  
                        // строки и помещается в пул  
                        // строк.  
String name2 = "Вася"; // в переменную присваивается  
                        // ссылка на ранее созданный  
                        // объект.
```

При создании объекта строки оператором **new** всегда создается новый объект.

¹ Последовательность символов начинающаяся с знака %

Статические методы класса *String*

- **valueOf** — возвращает строковое представление значений примитивных типов.

Пример:

```
String value = String.valueOf(3);
value = String.valueOf(0.5);
value = String.valueOf(true);
```

- **format** — метод возвращает строку, в которой спецификаторы формата¹ заменены на значения параметров в методе.

Синтаксис:

```
String.format(<строка шаблона>,
              <подставляемые значения>);
```

Пример:

```
String result = String.format("Жили у бабули
                              %d веселых гуся", 2);
```

При форматировании используются спецификаторы формата:

Специ- фикатор формата	Выполняемое форматирование
%a	Шестнадцатеричное значение с плавающей точкой
%b	Логическое (булево) значение аргумента

¹ Квадратным называют двухмерный массив у которого количество элементов в вложенных массивах равно количеству элементов в исходном массиве.

Специ- фикатор формата	Выполняемое форматирование
%c	Символьное представление аргумента
%d	Десятичное целое значение аргумента
%h	Хэш-код аргумента
%e	Экспоненциальное представление аргумента
%f	Десятичное значение с плавающей точкой
%g	Выбирает более короткое представление из двух: %e или %f
%o	Восьмеричное целое значение аргумента
%n	Вставка символа новой строки
%s	Строковое представление аргумента
%t	Время и дата
%x	Шестнадцатеричное целое значение аргумента
%%	Вставка знака %

Количество спецификаторов формата должно соответствовать количеству параметров в методе. Тип значений должен соответствовать спецификатору формата.

Escape-последовательности

Некоторые символы нельзя вставлять в строку (например двойные кавычки), для таких символов применяется экранирование с помощью косой черты (прямой слэш).

Пример:

```
String text = "\"Символ, которому предшествует  
наклонная черта влево (\\), является  
escape-последовательностью.\"";
```

Escape-последовательность	Описание
<code>\t</code>	Символ табуляции.
<code>\b</code>	Символ Backspace.
<code>\n</code>	Новая строка.
<code>\r</code>	Возврат каретки.
<code>\f</code>	Перевод формата.
<code>\'</code>	Символ одинарной кавычки.
<code>\»</code>	Символ двойной кавычки.
<code>\\</code>	Символ наклонной черты влево.

Методы экземпляра

Все методы класса `String` не изменяют строку, у которой они вызываются, а возвращают ссылку на новый объект класса `String`.

Неизменяемость (*immutable*) объекта строки обусловлена:

- **Конструкцией языка (Design).** Строки создаются в специальной области памяти в Java куче, известной как «String Intern pool». Когда Вы создаете новую строку с использованием строкового литерала (не с помощью конструктора или каких либо библиотечных функций), проверяется существует ли уже объект такой строки в строковом пуле. Если строка существует в пуле, то возвращается ссылка на уже существующий объект, иначе создается новый объект строки в пуле и возвращается ссылка на него. Если строку можно было бы изменить, работа строкового пула была бы не возможна.

- **Безопасность** (*Security*). Строки широко используются в качестве параметров для открытия файлов и соединений к базам данных, подключения к сетевым ресурсам, в качестве паролей и т.д. Если бы строку можно было бы изменить во всех этих случаях — это бы привело к серьезным угрозам безопасности программы и данных.
- **Эффективность** (*Efficiency*). Хэш-код строки часто используется в java в качестве ключа, например в реализации ассоциативного массива `java.util.HashMap`. Неизменяемость строки гарантирует нам, что ее хэш-код всегда будет одинаковым, что позволяет рассчитывать хэш-код один раз, во время создания строки, что при частом использовании существенно ускоряет работу с строками.
- **concat** — возвращает конкатенация (объединение двух строк в одну).

Пример:

```
String firstName= "Иван";
String lastName= " Иванов";
String fullname = fullname = cat.concat(name); //
// аналогично
// firstName+ " " + lastName;
System.out.println(fullname);
```

- **length** — возвращает количество символов в строке.
- **isEmpty** — возвращает истину, если строка не содержит символов, иначе ложь. Работает быстрее чем `length`.
- **charAt** — возвращает символ из строки по индексу.

Пример:

```
String testString = "test";
char myChar = testString.charAt(3);
System.out.println(myChar);
```

Результат: t.

- **contains** — возвращает истину, если строка содержит хотя бы одно совпадение с сравниваемой строкой.

Пример:

```
String testString = "testing";
boolean test = testString.contains("st");
System.out.println(test);
```

Результат: true.

- **startsWith** — возвращает истину, если строка начинается с искомого символа или строки.

Пример:

```
String str1 = "Star Wars";
boolean test = str1.startsWith("Star");
System.out.println(test);
```

Результат: true.

- **endsWith** — возвращает истину, если строка заканчивается на с искомый символ или строку.

Пример:

```
String str1 = "java.exe";
boolean test = str1.endsWith(".exe");
System.out.println(test);
```

Результат: true.

- **trim** — возвращает строку с удаленными начальными и конечными пробелами.
- **toLowerCase** — возвращает строку в которой все заглавные символы исходной строки заменены на строчные.
- **toUpperCase** — возвращает строку в которой все строчные символы исходной строки заменены на заглавные.
- **indexOf** — возвращает индекс символа, с которого найдено первое совпадение с искомой строкой или символом. Поиск начинается с начала строки. Если совпадение не найдено возвращает -1;

Пример:

```
String text = "polymorphism";  
int index = text.indexOf('o');  
System.out.println(index);
```

Результат: 1.

Метод **indexOf** позволяет начать поиск не с начала, а с определенного символа.

Пример:

```
String text = "polymorphism";  
int index = text.indexOf('o');  
index=text.indexOf('o',index+1);  
System.out.println(index);
```

Результат: 5.

- **lastIndexOf** — возвращает индекс символа, с которого найдено первое совпадение с искомой строкой или

символом. Поиск начинается с конца строки. Если совпадение не найдено возвращает -1;

Пример:

```
int index = "readme.txt".lastIndexOf(".");  
System.out.println(index);
```

Результат: 6.

- **substring** — возвращает часть строки из исходной.

Пример:

```
// вырезает строку с 6 символа включительно и до  
// конца строки.  
String world = "Hello World".substring(6);  
System.out.println(world);
```

Результат: World.

Пример:

```
String world = "Hello World".substring(1, 4);  
System.out.println(world);
```

Результат: ell.

Примечание. Метод *substring* возвращает новую строку, но массив с символами берется из исходной строки, изменяется только индекс смещения с начала массива (*offset*) и количество элементов от индекса смещения (*count*), в связи с этим сборщик мусора не сможет очистить память от неиспользуемой части строки.

- **replace** — возвращает строку, заменяя в исходной строке символ или набор символов на другой символ или набор символов.
- **getBytes** — возвращает строку в виде массива байт.
- **getChars** — извлекает символы строки в массив символов;
- **split** — поиск вхождения в строку заданного регулярного выражения (разделителя) и деление исходной строки в соответствии с этим на массив строк.

Массивы строк

Подобно существованию массивов объектов любого другого типа, могут существовать и массивы символьных строк. В частности, массивом строк является аргумент **String args[]** метода **main()**. Когда программа вызывается из командной строки, все строки (разделенные пробелами), которые записываются после имени программы, передаются в нее как значения элементов этого массива.

В следующем примере программы выводятся все аргументы командной строки, с которыми она вызывается:

```
public class TestCommandLine {  
    public static void main(String[] args)  
    {  
        for(int i=0;i<args.length;i++)  
            System.out.println("arg["+i+"]= "+args[i]);  
    }  
}
```

После вызова из командной строки:

```
c:\....>java TestCommandLine One Two Three
```

ВЫВОДИТСЯ:

```
arg[0]= One
arg[1]= Two
arg[3]= Three
```

Класс *StringBuffer*

Как уже упоминалось ранее, внутренние данные объекта класса *String* нельзя изменить после его создания, поэтому для работы со строками как с изменяемой структурой применяется класс *StringBuffer*. В отличие от строк, объекты класса *StringBuffer* нельзя создавать с помощью строковых литералов, только с помощью оператора *new*.

Пример:

```
StringBuffer sb = "test"; // ошибка компиляции
StringBuffer sb = new StringBuffer("test");
```

Методы класса *StringBuffer*

- **append** — добавляет новую строку в объект класса *StringBuffer* и возвращает ссылку на этот же объект.

Пример:

```
StringBuffer sb = new StringBuffer("test"); //
                                           // создание объекта
sb.append('-').append("test"); // добавление значений
                                           // цепочкой
```

```
sb.append(true); // единичное добавление
sb.append(1);
System.out.println(sb);
```

Результат: test-testtrue1.

Примечание. Конкатенация строк в цикле объектов класса *String* с использованием оператора `+` может привести к проблемам с производительностью в связи с тем, что постоянно создаются новые объекты для строк. В такой ситуации предпочтительнее использовать объект класса *StringBuffer* или *StringBuilder*.

- **insert** — вставляет строку или символ в объект класса *StringBuffer*.

Пример:

```
StringBuffer sb = new StringBuffer("I java!");
sb.insert(2, "love "); // 2 — индекс символа после
                      // которого будет вставлена
                      // строка
System.out.println(sb.toString());
```

Результат: I love java!

- **reverse** — меняет порядок символов на обратный.

Пример:

```
StringBuffer sb = new StringBuffer("palindrome");
sb.reverse();
System.out.println(sb);
```

Результат: emordnilap.

- **delete** — удаляет часть строки начиная с указанного индекса.

Пример:

```
StringBuffer phrases = new StringBuffer("I do not  
like java!");  
phrases.delete(2, 7);  
System.out.println(phrases);
```

- **deleteCharAt** — удаляет символ из строки, по указанному индексу.
- **replace** — заменяет подстроку в строке.

StringBuilder, его отличия от класса StringBuffer

Класс `StringBuilder` имеет точно такое же предназначение и такие же методы, как и `StringBuffer`. Класс `StringBuilder` отличается от `StringBuffer` отсутствием синхронизации данных для много-поточного программирования, поэтому его использование в однопоточной программе предпочтительнее.

Метод toString()

Все классы в java неявно наследуются от класса `Object`. В классе `Object` имеется метод `toString()`, который возвращает строковое представление состояния объекта.

В классах `StringBuilder` и `StringBuffer` этот метод возвращает объект класса `String`, который содержит строку хранящуюся в объекте классов `StringBuilder` или `StringBuffer` на момент вызова метода.

Класс *StringTokenizer*

Класс *StringTokenizer* предназначен для разбиения строки на части (токены). В качестве разделителя используется регулярное выражение переданное вторым параметром в конструктор класса.

```
String s = "Best Java programming language.";
StringTokenizer st = new StringTokenizer(s);

while (st.hasMoreTokens())
{
```

Пример разбиения строки на слова, используя в качестве разделителя символ пробела:

```
System.out.println(st.nextToken());
}
```

Пример разбиения строки на слова, используя в качестве разделителя запятую:

```
String s = "Best,Java,programming,language.";
StringTokenizer st = new StringTokenizer(s, ",");

while (st.hasMoreTokens())
{
    System.out.println(st.nextToken());
}
```

Особенности конкатенации строк с другими типами

Строки полученные в результате конкатенации строковых литералов и переменных в пул строк не помещаются.

Пример:

```
String t = "t";
String cat1 = "ca" + t;
String cat2 = "cat";
System.out.println(cat1 == cat2);
```

Результат: false.

В случае конкатенации строк оператором + с другими примитивными типами данных, все последующие типы преобразуются в строчное представление. В связи с этим можно получить не то, что ожидается.

Пример:

```
String text = "sum =" + 1 + 2; // 1 и 2 преобразуются
                               // в строку
System.out.println(text );
```

Результат: sum =12.

Добавление круглых скобок в выражение меняют порядок выполнения. Сначала произойдет операция сложения литералов и только потом преобразование к строке.

Пример с скобками:

```
String text = "sum =" + (1 + 2);
System.out.println(text );
```

Результат: sum =3.

Пример:

```
String text = 1 + 2 + " = 3";
System.out.println(text );
```

Результат: 3 = 3.

Пример с комбинацией операторов:

```
String text = "mul =" + 3 * 2;
System.out.println(text );
```

Результат: mul =6.

Сравнение строк

При применении оператора сравнения (==) для переменных ссылочного типа происходит сравнение ссылок на объект хранящихся в переменных, в связи с этим если создать две строки с идентичным содержанием, через оператор **new**, сравнение таких строк оператором == вернет **false**. В связи с этим для сравнение идентичности разных строк следует использовать метод **equals**, который сравнивает строки на эквивалентность.

Пример сравнения строк инициализированных литералами:

```
String text1 = "java";
String text2 = "java"; // в переменную присваивается
                        // ссылка на объект из пула
                        // строк
System.out.println(text1 == text2);
System.out.println(text1.equals(text2));
```

Результат: true true.

Пример сравнения строк созданных через оператор **new**:

```
String text1 = new String("java");
String text2 = new String("java");
System.out.println(text1 == text2);
System.out.println(text1.equals(text2));
```

Результат: false true.

Строку созданную с помощью конкатенации или оператора `new` можно программно поместить в «пул строк» используя метод `intern()`. Метод `intern()` проверяет есть ли данная строка в пуле строк и если такая строка отсутствует помещает ее в пул строк. Возвращает ссылку на строку из пула строк.

Пример:

```
String text1 = new String("java");

text1 = text1.intern();
text2 = text2.intern();

String text2 = new String("java");
System.out.println(text1 == text2);
System.out.println(text1.equals(text2));
```

Результат: *true true.*

ВАЖНО!!! В классах `StringBuilder` и `StringBuffer` метод `equals` не переопределен и следуя стандартной реализации в классе `Object` сравнивает ссылки на объекты. В связи с этим при необходимости сравнить два объекта классов `StringBuilder` или `StringBuffer` преобразуйте оба объекта к строке методом `toString`.

Пример:

```
StringBuffer sb1 = new StringBuffer("abc");
StringBuffer sb2 = new StringBuffer("abc");
System.out.println(sb1.equals(sb2)); // неверно
```

Результат: *False.*

Пример.

С консоли вводится строка, содержащая несколько слов, разделенных пробелами. Пробелы могут быть также перед первым и после последнего слова. Нужно удалить лишние пробелы в начале и в конце строки, а между словами оставить ровно один пробел.

```
public class Runner {
    public static void main(String[] args) {
        String data;
        System.out.println("Enter source string");
        Scanner scan=new Scanner(System.in);
        data=scan.nextLine();
        data=data.trim(); // удаление пробелов в
                          // начале и в конце
        String[] words=data.split(" + "); //
        // разделение на слова собрать из слов строку
        // с одним пробелом между
        StringBuffer sb=new StringBuffer(words[0]);
        for(int i=1;i<words.length;i++){
            sb.append(" ").append(words[i]);
        }
        System.out.println("|"+sb.toString()+"|");
    }
}
```

Пример результата работы программы:

```
Enter source string
   I           like           Java
|I like Java|
```

Пример.

Пользователь вводит строку, представляющую собой дату в формате dd/mm/yyyy. Вывести на экран дату, указав

название месяца словами. Не использовать специальные классы для работы с датами! Примерный результат работы программы:

```
Enter the date: 16/03/2007
16 march 2007y.
```

Решение задачи:

```
import java.util.Scanner;
public class Runner {
    public static void main(String[] args) {
        String s;
        System.out.print("Enter the date: ");
        Scanner scan=new Scanner(System.in);
        s=scan.nextLine();

        // разделение строки на части
        String[] parts=s.split("/");

        //преобразование строки в месяц
        int month;
        month=Integer.parseInt(parts[1]);

        //получить название месяца
        String monthStr=monthName(month);

        // собрать новую строку и вывести
        s=parts[0]+" "+ monthStr+" "+parts[2]+"y.";
        System.out.println(s);
    }

    static String monthName(int num) {
        switch (num) {
            case 1: return "january";
            case 2: return "february";
            case 3: return "march";
            case 4: return "april";
            case 5: return "may";
            case 6: return "june";
        }
    }
}
```

```

        case 7: return "july";
        case 8: return "august";
        case 9: return "september";
        case 10: return "oktober";
        case 11: return "november";
        case 12: return "december";
        default: return "unknown";
    }
}

```

Регулярные выражения

«Некоторые люди, столкнувшись с проблемой, думают: — «Я знаю как ее решить. Я буду использовать регулярные выражения». Теперь у таких людей две проблемы».¹

Регулярные выражения² — это формальный язык, предназначенный для поиска и различных манипуляций с подстроками текста. Основой языка являются метасимволы. Регулярная строка является образцом или шаблоном, который задает правила поиска подстрок в исходной строке.

Регулярные выражения являются мощным и гибким инструментом для работы с текстом, в разы уменьшают количество исходного кода, но использование регулярных выражений часто усложняет понимание и чтение кода. Поэтому рекомендуется очень тщательно продумывать необходимость использования регулярных выражений.

¹ Цитата принадлежит Джейми Завински (Jamie Zawinski, JWZ) одному из первых разработчиков Netscape.

² Регулярные выражения (англ. regular expressions) в среде программистов часто называют регулярками и регэксп (жаргонное).

Большинство символов в регулярных выражениях представляют самих себя, кроме метасимволов.

Основные метасимволы регулярных выражений:

Мета символ	Назначение
<code>^</code>	Начало проверяемой строки
<code>\$</code>	Конец проверяемой строки
<code>.</code>	Сокращенная форма записи для символьного класса, совпадающего с любым символом
<code> </code>	Подвыражения, объединенные этим способом, называются альтернативами (alternatives)
<code>?</code>	Предшествующий ему символ является необязательным
<code>+</code>	Обозначает «один или несколько экземпляров непосредственно предшествующего элемента»
<code>*</code>	Любое количество экземпляров элемента (в том числе и нулевое)
<code>\\d</code>	Цифровой символ
<code>\\D</code>	Не цифровой символ
<code>\\s</code>	Пробельный символ
<code>\\S</code>	Любой символ, кроме пробела
<code>\\W</code>	Любой символ, кроме буквенного или цифрового символа или знака подчёркивания
<code>\\w</code>	Буквенный или цифровой символ или знак подчёркивания

В классе `String` есть несколько методов использующих регулярные выражения:

- **`matches`** — метод возвращает истину если исходная строка соответствует регулярному выражению, иначе возвращает ложь.

Пример проверяет соответствует ли строка правилам составления электронного адреса:

```
String email = "unguryan@itstep.org";
// регулярное выражение для проверки правильности
// email адреса
String regular = "[a-zA-Z]{1}[a-zA-Z\\d\\u002E\\u005F]+@([a-zA-Z]+\\u002E){1,2}((net)|(com)|(org))";
System.out.println(email.matches(regular));
```

Результат: true.

- **replaceAll** — возвращает строку в которой все подстроки удовлетворяющие условию регулярного выражения заменены на строку из второго параметра.

Пример:

```
String text= "I love coffee!";
text = text.replaceAll("[Cc]offee", "java");
```

- **split** — Возвращает массив строк, разбивая строку на части используя в качестве разделителя регулярное выражение.

В Java есть пакет `java.util.regex`, предназначенный для работы с регулярными выражениями.

Пример:

```
String url= "http://mystat.itstep.org/";
// регулярное выражение для проверки правильности
// URL адреса
String regular = "^(?:([a-z]+):(?:([a-z]*)?)?\\/\\"/>

```

```

(?: (?: [^:@]* ) (?: (?: [^:@]* ) ? @ ) ? ( (?: [a-z0-9_-]+ \. )+ [a-z]
{2,} | localhost | (?: (?: [01]? \d \d ? | 2 [0-4] \d | 25 [0-5] ) \. )
{3} ) (?: (?: [01]? \d \d ? | 2 [0-4] \d | 25 [0-5] ) ) ) (?: (?: \d+ ) ?
(?: (?: [^:\? \# ]+ ) ) ? (?: \? (?: [^ \# ]+ ) ) ? (?: \# (?: [^\s ]+ ) ) ? $ / i";
Pattern p = Pattern.compile(regular); // создает
                                         // объект
                                         // образец
Matcher m = p.matcher(url); // создает объект
                               // совпадений
System.out.println(m.matches());

```

Результат: true.

JShell

JShell — это консольное приложение, которое появилось в JDK 9. Оно позволяет вводить части программы и сразу же наблюдать результат их выполнения. С его помощью можно протестировать отдельные фрагменты кода, попробовать разные варианты использования метода или познакомиться с интерфейсом незнакомого класса. То есть приложение JShell дает возможность изучать язык Java «методом научного тыка»!

JShell не заменяет, а дополняет среду разработки (IDE). Протестированный в JShell код можно затем сохранить в файле или вставить в текст программы в редакторе IDE. Конечно, особенно удобно это приложение при разработке программы без IDE, в обычном текстовом редакторе.

Чтобы можно было использовать JShell, на компьютере должна быть установлена версия JDK 9.0 (или старше).

Для запуска приложения нужно задать команду `jshell` в командной строке. Текущей директорией должна быть директория `bin` загруженного JDK (рис. 39).

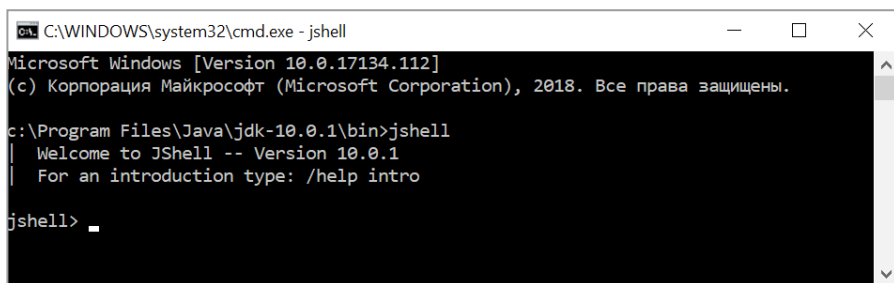


Рисунок 39.

Команды JShell начинаются с символа /. Для завершения работы с JShell используется команда /exit. Список всех команд можно вывести на экран, задав команду /? (рис. 40).

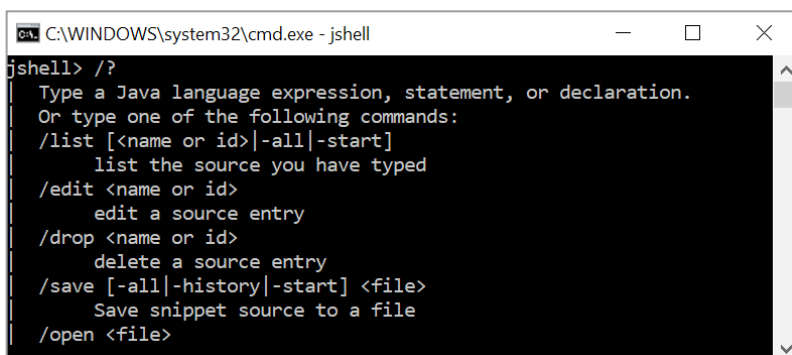


Рисунок 40.

Фрагменты кода, которые пользователь вводит в JShell, называются снippetами (*snippets*). Снippetы выполняются сразу после ввода. При этом JShell показывает результаты, описывает выполненные действия и возникающие ошибки.

Степень «разговорчивости» программы определяет параметр *feedback* (*обратная связь*). По умолчанию этот параметр имеет значение *normal*. Для начинающего

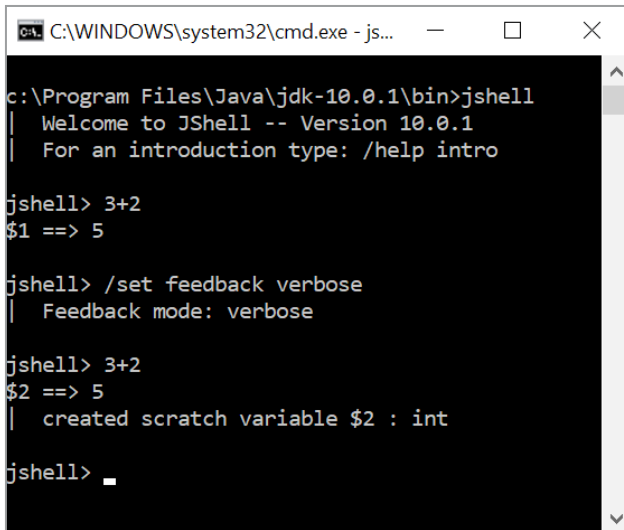
пользователя лучше установить максимальную обратную связь, задав этому параметру значение `verbose` командой

```
/set feedback verbose
```

Можно также начать сеанс работы в JShell с этого режима, если запустить программу с ключом `-v`:

```
c:\Program Files\Java\jdk-10.0.1\bin>jshell -v
```

Различия используемого параметра обратной связи продемонстрированы на рисунке 41.



```

C:\WINDOWS\system32\cmd.exe - js...
c:\Program Files\Java\jdk-10.0.1\bin>jshell
| Welcome to JShell -- Version 10.0.1
| For an introduction type: /help intro

jshell> 3+2
$1 ==> 5

jshell> /set feedback verbose
| Feedback mode: verbose

jshell> 3+2
$2 ==> 5
| created scratch variable $2 : int

jshell>

```

Рисунок 41.

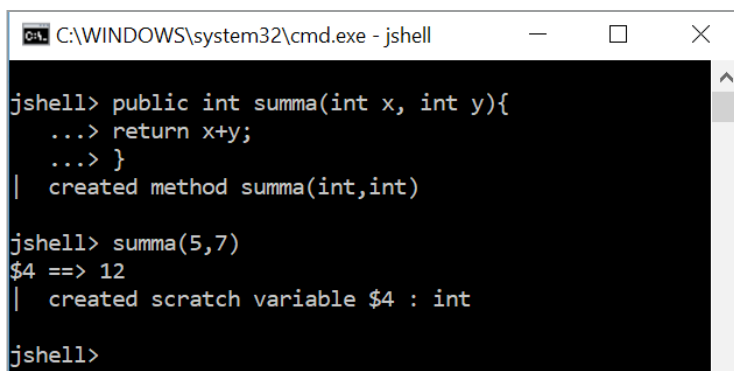
В первом случае (режим `normal` по умолчанию) JShell только показывает имя и значение скретч-переменной, которая была создана для размещения значения выражения `3+2` (имена, созданные автоматически в JShell начинаются со знака `$`).

Во втором случае подробно прокомментированы действия: «создана скретч-переменная \$2 типа int».

Полная противоположность режиму `verbose` — режим `silent`. При этом программа вообще ничего не будет комментировать.

Приятный бонус: если пользователь забывает поставить точку с запятой в конце сниппета, то JShell сама ее добавит.

Можно вводить не только однострочные фрагменты. Если программа понимает, что код еще не завершен, то в начале строки выводится символ продолжения `...>`. Пример объявления и тестирования метода:



```
C:\WINDOWS\system32\cmd.exe - jshell

jshell> public int summa(int x, int y){
...> return x+y;
...> }
| created method summa(int,int)

jshell> summa(5,7)
$4 ==> 12
| created scratch variable $4 : int

jshell>
```

Рисунок 42.

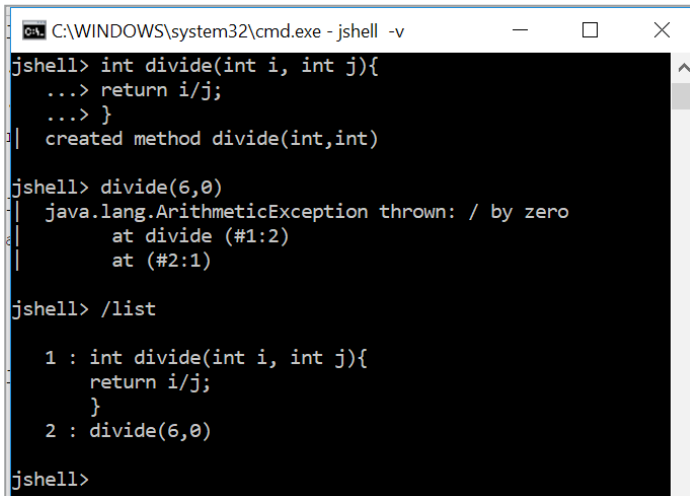
Если после тестирования кода пользователю стало ясно, что описание переменной, метода или класса неверное, то его легко изменить повторным вводом описания.

В JShell есть интересная возможность использовать «будущие ссылки» (*forward references*). Если при описании метода используются переменные, методы или классы, которые еще не объявлены, то код все равно будет при-

нут. При этом будет выдано предупреждение о том, что использовать такое описание можно будет только после добавления недостающих фрагментов кода.

JShell отслеживает моменты возникновения исключений и выводит информацию о них в виде `#ID:line-number`, где `ID` — номер сниппета, а `line-number` — номер строки в сниппете.

Список введенных ранее сниппетов можно вывести командой `/list`. Пример определения исключения в коде:



```

C:\WINDOWS\system32\cmd.exe - jshell -v
jshell> int divide(int i, int j){
...> return i/j;
...> }
| created method divide(int,int)

jshell> divide(6,0)
| java.lang.ArithmeticException thrown: / by zero
|   at divide (#1:2)
|   at (#2:1)

jshell> /list

  1 : int divide(int i, int j){
    return i/j;
    }
  2 : divide(6,0)

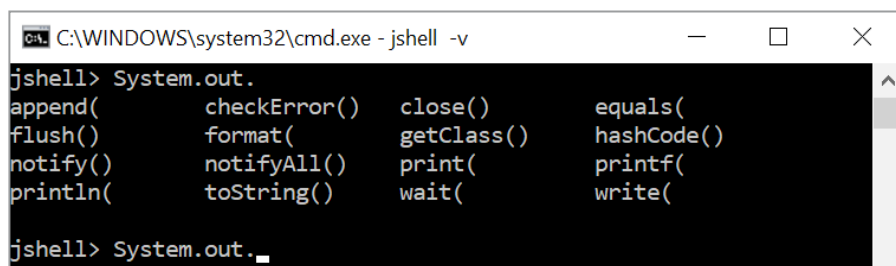
jshell>
  
```

Рисунок 43.

При вводе сниппетов можно использовать клавишу `Tab` для автоматического завершения (дописывания) кода. Если имеется несколько вариантов завершения, то JShell покажет их на экране.

Таким способом можно также изучать методы нового для пользователя класса. На рисунке ниже клавиша `Tab` была нажата после точки. В результате были выведены

все методы, которые можно вызывать от имени объекта `System.out`.

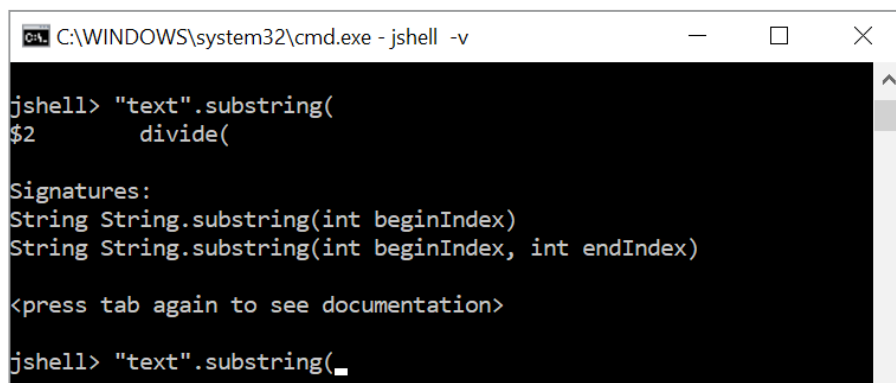


```

C:\WINDOWS\system32\cmd.exe - jshell -v
jshell> System.out.
append(      checkError()  close()      equals(
flush()      format(      getClass()   hashCode()
notify()     notifyAll()  print(       printf(
println(     toString()   wait(        write(
jshell> System.out._
  
```

Рисунок 44.

Аналогично можно попросить подсказку о возможных параметрах метода. На рисунке показаны все перегрузки метода `substring()` (клавиша `Tab` нажата после открывающей скобки).



```

C:\WINDOWS\system32\cmd.exe - jshell -v
jshell> "text".substring(
$2      divide(

Signatures:
String String.substring(int beginIndex)
String String.substring(int beginIndex, int endIndex)

<press tab again to see documentation>
jshell> "text".substring(_
  
```

Рисунок 45.

Прочие возможности JShell можно изучить на сайте Oracle по ссылке: <https://docs.oracle.com/javase/10/jshell/introduction-jshell.htm#JSHEL-GUID-630F27C8-1195-4989-9F6B-2C51D46F52C8>.

Домашняя работа

1. Организовать ввод с клавиатуры даты рождения человека, программа должна вывести знак зодиака и название года по японскому календарю. Предполагаем, что пользователь всегда корректно вводит данные.

Пример входных данных:

5 декабря 1974 г

Вывод:

Знак: стрелец Год: тигра

2. Программа запрашивает шестизначное число, после ввода определяет будет ли являться счастливым билет с таким номером (сумма первых трех цифр совпадает с сумой трех последних).

Пример входных данных:

421151

954832

Вывод:

Да

Нет

Показать на консоли все [числа Армстронга](#) в диапазоне от 0 до 10.000.000. Число Армстронга, или Самовлюблённое число, или совершенный цифровой инвариант или — натуральное число, которое равно сумме своих цифр, возведённых в степень, равную количеству его цифр. Например: $13 + 53 + 33 = 153$.

3. Необходимо нарисовать ёлочку символом звёздочки. Каждый новый ярус должен быть шире предыдущего.

С клавиатуры вводится количество ярусов, и высота первого (верхнего) яруса ёлочки. Ёлочка должна быть симметричная.

Пример работы программы:

```
Введите число ярусов и высоту через пробел:
3 4
```

```

      *
     ***
    *****
   *********
  *
  ***
  *****
 *******
*****
 *
 ***
 *****
 *******
*****
 *
 ***
 *****
 *******
*****

```

4. Сформируйте массив из 10 элементов, заполнив его целыми случайными числами от -10 до 10 . Определите среднее арифметическое элементов массива. Сформируйте новый массив, переписав в него элементы исходного массива, меньшие среднего. Выполните сортировку исходного массива по возрастанию элементов.
5. Сформируйте двумерный целочисленный массив. Число строк и столбцов нужно запросить у пользователя. Заполните массив случайными числами от 0

до 10 и выведите его на консоль. Удалите из массива одну строку (номер строки запросите у пользователя). Распечатайте полученный массив. Используйте статические методы для реализации каждого действия (инициализация массива, вывод на консоль, удаление строки).